

TRABAJO FIN DE MÁSTER

**Un entorno para la creación de lenguajes de dominio específico
con sintaxis basada en bloques**

Máster Universitario en

Sistemas Interactivos Inteligentes

Autor: Xiaoyong Wang

Tutores: Juan de Lara y Pablo Gómez Abajo
Departamento de Ingeniería Informática

FECHA: 25 de junio de 2026
Curso académico: 2025–2026

Resumen

Este Trabajo Fin de Máster aborda la creación de lenguajes de dominio específico con sintaxis basada en bloques. Bibliotecas como Blockly facilitan la creación de editores visuales, pero todavía que el desarrollador defina manualmente los bloques, los campos, las conexiones, la paleta, las validaciones y los generadores en JavaScript. Este esfuerzo debe repetirse cada vez que se necesita un editor para un nuevo dominio.

Para reducir este esfuerzo, se propone Model2Blockly, un entorno generativo distribuido como plugin de Eclipse y como cadena de generación web. La herramienta admite dos rutas de entrada: metamodelos Ecore enriquecidos con anotaciones y descripciones textuales .m2b definidas con Xtext. La ruta es un DSL de definición de lenguajes para describir qué bloques, campos, conexiones y validaciones debe tener el editor generado. Ambas rutas se normalizan en un metamodelo EMF intermedio, EditorSpec, que describe bloques, campos, categorías, entradas de valor, referencias, validaciones y metadatos de interfaz. Sus instancias se serializan como XMI y se recargan como contrato entre la transformación de modelos y la generación final. A partir de ese modelo se generan editores HTML/JavaScript que pueden abrirse en el navegador y permiten guardar, cargar, exportar JSON y XMI de instancia, mostrar avisos de validación e inspeccionar las reglas como bloques.

La solución se evalúa mediante un ejemplo conductor, AppMaker, descrito tanto como metamodelo Ecore anotado como mediante el DSL textual .m2b. Este dominio cubre pantallas, widgets, acciones, navegación, fuentes de datos, contenciones, herencia, clases abstractas, referencias dinámicas, campos obligatorios, validaciones simples y plantillas de código. La memoria compara ambas formas de autoría y muestra cómo convergen en el mismo modelo intermedio antes de generar el editor Blockly. El proyecto incluye un update site p2 público, informes de generación, editores desplegados en GitHub Pages y 142 pruebas JUnit organizadas por capas. Los resultados indican que Model2Blockly permite generar editores Blockly a partir de descripciones de dominio de alto nivel, reduce la programación manual

y mantiene una arquitectura MDE reutilizable.

Palabras clave: lenguajes de dominio específico, programación basada en bloques, Blockly, EMF, Ecore, Xtext, ingeniería dirigida por modelos.

Abstract

This Master's Thesis addresses the creation of domain-specific languages with a block-based syntax. Libraries such as Blockly help developers build visual editors, but they still require developers to define blocks, fields, connections, toolboxes, validations and generators manually in JavaScript. This work must be repeated whenever an editor is needed for a new domain.

To reduce this effort, the thesis proposes Model2Blockly, a generative environment distributed as an Eclipse plugin and as a web generation pipeline. The tool supports two input routes: annotated Ecore metamodels and textual `.m2b` descriptions defined with Xtext. The textual route is a language-definition DSL for specifying the blocks, fields, connections and validations of the generated editor. Both routes are normalised into an intermediate EMF metamodel, `EditorSpec`, which describes blocks, fields, categories, value inputs, references, validations and interface metadata. Its instances are serialized as XMI and reloaded as the contract between model transformation and final generation. From this model, the tool generates HTML/JavaScript editors that can be opened in a browser and support saving, loading, JSON and domain instance XMI export, validation warnings, and inspection of validation rules as blocks.

The solution is evaluated through a running example, `AppMaker`, described both as an annotated Ecore metamodel and as a textual `.m2b` language definition. This domain covers screens, widgets, actions, navigation, data sources, containments, inheritance, abstract classes, dynamic references, required fields, simple validations and code templates. The thesis compares the two authoring forms and shows how they converge on the same intermediate model before generating the Blockly editor. In addition, the project includes a public p2 update site, generation reports, editors deployed through GitHub Pages, and 142 JUnit tests organised by architectural layer. The results indicate that Model2Blockly can generate Blockly editors from high-level domain descriptions, reduces manual programming and

keeps a reusable MDE architecture.

Keywords: domain-specific languages, block-based programming, Blockly, EMF, Ecore, Xtext, model-driven engineering.

Índice general

Resumen	1
Abstract	3
1 Introducción	8
2 Objetivos y alcance	12
2.1 Objetivo general	12
2.2 Objetivos específicos	13
2.3 Alcance de la solución	14
2.4 Límites del trabajo	16
2.5 Criterios de evaluación	16
3 Estado del arte y tecnologías	18
3.1 Programación basada en bloques	18
3.2 Blockly	20
3.3 Lenguajes de dominio específico e ingeniería dirigida por modelos	24
3.4 EMF y Ecore	25
3.5 Xtext	27
3.6 Xtend	28
3.7 Trabajos relacionados sobre DSLs basados en bloques	29
3.8 Comparación y posicionamiento de la propuesta	30
3.9 Relación entre las tecnologías	31
4 Diseño y arquitectura de la solución	33
4.1 Enfoque general	33

4.2	Flujo general de generación	34
4.3	Modelo intermedio	35
4.4	Ruta basada en Ecore	36
4.5	Ruta textual .m2b basada en Xtext	37
4.6	Generación de editores Blockly	37
4.7	Validaciones, referencias y personalización	38
4.8	Trazabilidad con los objetivos	39
5	Implementación de Model2Blockly	40
5.1	Ejecución de Model2Blockly	40
5.2	Flujo interno de una ejecución	43
5.3	Organización del proyecto	44
5.4	Carga y validación de las entradas	46
5.4.1	Carga de un metamodelo .ecore	46
5.4.2	Carga y validación de una entrada .m2b	46
5.4.3	Niveles de validación y comunicación de errores	47
5.5	Ruta textual .m2b con Xtext	48
5.6	Modelo intermedio EMF y vista Java	49
5.7	Adaptador desde Ecore	51
5.8	Adaptador desde la ruta textual .m2b	54
5.9	Generador HTML/JavaScript	56
5.10	Validaciones y referencias en tiempo de ejecución	59
5.11	Integración con Eclipse y distribución	61
5.12	Entradas independientes (standalone)	62
5.13	Pruebas de implementación	63
5.14	Resumen de implementación	64
6	Ejemplo conductor y evaluación	65
6.1	Estrategia de evaluación	65
6.2	Ejemplo conductor seleccionado	67
6.3	AppMaker por las dos rutas de entrada	70
6.4	Cobertura funcional	75
6.5	Verificaciones técnicas	77

6.6	Resultados frente a los objetivos	78
6.7	Correspondencia entre metamodelo y editor Blockly	79
6.8	Limitaciones de la evaluación	79
6.9	Conclusión de la evaluación	80
7	Conclusiones y trabajo futuro	81
7.1	Viabilidad del enfoque	81
7.2	Principales aportaciones	82
7.3	Conclusiones por objetivos	83
7.4	Limitaciones	84
7.5	Trabajo futuro	85
7.6	Cierre	86

Capítulo 1

Introducción

Crear sistemas, automatizar tareas y expresar ideas de un dominio mediante programación puede suponer una barrera importante para los usuarios principiantes. La literatura sobre programación basada en bloques muestra que estos entornos se utilizan a menudo para introducir a usuarios noveles en la programación, porque ofrecen una entrada más accesible que los lenguajes textuales tradicionales [1]. Esta dificultad no aparece exclusivamente en la enseñanza de la informática. También afecta a usuarios no expertos que podrían utilizar la programación para crear simulaciones, automatizar procesos o representar conocimiento de un dominio.

Los lenguajes basados en bloques reducen parte de esta dificultad. En lugar de escribir instrucciones textuales desde cero, el usuario trabaja con piezas visuales que se arrastran y se conectan. La forma de las piezas, sus colores, sus etiquetas y la organización por categorías ayudan a reconocer qué construcciones son posibles [2]. Herramientas como Scratch, App Inventor, Tynker o MakeCode muestran esta idea en contextos educativos y en dominios como robótica, ciencia de datos, desarrollo móvil o creación de juegos [1].

Blockly es una de las bibliotecas más usadas para crear editores de este tipo. Su documentación lo describe como una biblioteca web que permite integrar en una aplicación un editor visual basado en bloques. Blockly se ocupa de la representación visual, el arrastre, el encaje de piezas y la gestión del workspace. El desarrollador, en cambio, debe definir los bloques, los campos, las conexiones y la generación asociada [3, 4].

Por lo tanto, Blockly facilita la creación de editores visuales, pero no suprime todo el esfuerzo necesario. Cada nuevo dominio sigue necesitando tipos de bloque, campos, entradas, categorías de toolbox, reglas de conexión, serialización, validaciones y, en algunos casos,

generación de código. Este esfuerzo debe repetirse cuando se quiere crear un editor de bloques para otro dominio. Además, los entornos de programación por bloques no siguen un único diseño. La literatura indica que cada entorno toma decisiones propias sobre la organización de los bloques, la relación con el código textual y la transición entre programación visual y textual [1, 2].

Esta situación plantea un problema de ingeniería de lenguajes. Si los conceptos principales de un dominio ya están descritos en un modelo o metamodelo, no debería ser necesario duplicar la misma información como código JavaScript de Blockly. La ingeniería dirigida por modelos propone precisamente trabajar con modelos como artefactos principales para especificar, diseñar, probar y generar software [5]. También se han señalado motivaciones como la reutilización, la reducción del tiempo de desarrollo y la mejora de calidad [6, 7]. Desde esta perspectiva, un editor Blockly puede verse como una sintaxis visual generada a partir de una descripción abstracta del dominio.

El ecosistema Eclipse ofrece tecnologías adecuadas para explorar esta idea. EMF permite definir modelos estructurados y generar código a partir de ellos. Su metamodelo Ecore representa clases, atributos, referencias, herencia y cardinalidades [8, 9]. Xtext permite definir lenguajes textuales de dominio específico y genera infraestructura de parsing, enlace, validación y edición [10, 11]. Estas tecnologías permiten describir dominios con precisión, pero no generan automáticamente una sintaxis visual basada en bloques.

El objetivo de este Trabajo Fin de Máster es desarrollar un entorno para crear lenguajes de dominio específico con sintaxis basada en bloques. La idea no es construir un único editor de bloques, sino una infraestructura que permita describir un dominio y generar de forma automática el código necesario para editarlo con Blockly.

La herramienta desarrollada se llama Model2Blockly. Se entrega como un conjunto de plugins Eclipse y como una cadena de generación HTML/JavaScript. Admite dos rutas de entrada. La primera parte de metamodelos Ecore, que pueden enriquecerse con anotaciones sobre la sintaxis visual. En esta ruta, las clases se convierten en tipos de bloque, los atributos en campos, las referencias de contención en entradas estructurales y las cardinalidades en reglas de comprobación. La segunda ruta parte de una sintaxis textual .m2b definida con Xtext. Esa sintaxis es un DSL de definición de lenguajes: permite al diseñador del lenguaje declarar qué bloques, campos, conexiones, categorías y validaciones tendrá el editor Blockly generado.

Ambas rutas se transforman en un modelo intermedio común, BlocklyEditorSpec.

Este modelo contiene los elementos necesarios para generar el editor: tipos de bloque, categorías, campos, entradas de valor, referencias, validaciones y metadatos de interfaz. La instancia intermedia se serializa como XMI y se recarga antes de producir los artefactos finales. A partir de esta representación se genera un editor Blockly en HTML/JavaScript que puede ejecutarse directamente en el navegador, junto con un informe de generación, un modelo de ejemplo y una vista visual de las reglas de validación.

El desarrollo se ha organizado de forma incremental. Primero se generaron bloques y paletas a partir de modelos de dominio. Después se añadieron validaciones para detectar campos obligatorios vacíos o bloques requeridos que faltan. También se incorporaron referencias dinámicas mediante listas desplegables, opciones de personalización visual, anotaciones de interfaz, referencias opuestas, validaciones semánticas simples, generación de código y exportación del modelo creado por el usuario como JSON o XMI de instancia. La herramienta se evalúa mediante AppMaker, un ejemplo conductor descrito por las dos rutas: como metamodelo Ecore anotado y como fichero textual .m2b. Esta doble descripción permite comparar la autoría en Ecore y en el DSL textual, y mostrar que ambas entradas se normalizan hacia el mismo modelo intermedio antes de generar el editor Blockly.

Las aportaciones principales del TFM son las siguientes. En primer lugar, se define una arquitectura que conecta Ecore, un modelo intermedio EMF y Blockly mediante transformaciones explícitas y serialización XMI. En segundo lugar, se implementa el adaptador Ecore que extrae clases, atributos, referencias, cardinalidades y anotaciones. En tercer lugar, se implementa una ruta textual .m2b con Xtext y un adaptador que transforma ese modelo al mismo EditorSpec. En cuarto lugar, se desarrolla un generador de editores Blockly con bloques, toolbox, páginas HTML, serialización, validaciones, exportación JSON/XMI e informes de generación. En quinto lugar, se añaden referencias, personalización visual, generación de código y una representación editable de reglas de validación que puede aplicarse de vuelta a la fuente. Por último, se evalúa la solución con un ejemplo conductor, pruebas automáticas y artefactos desplegados públicamente.

El alcance del trabajo se centra en la generación de editores Blockly desde descripciones de dominio de alto nivel, ya sean metamodelos Ecore anotados o modelos textuales .m2b, y en la reutilización de la arquitectura mediante un modelo intermedio común. Quedan fuera del objetivo principal la importación genérica de modelos XMI existentes, la integración completa de restricciones OCL arbitrarias y la creación de interfaces finales muy especializadas para cada dominio. La contribución central es reducir el esfuerzo necesario para crear

editores Blockly específicos mediante metamodelos, DSL textual, transformaciones de modelos y generación de código.

El resto del documento se organiza de la siguiente manera. El capítulo 2 presenta los objetivos, el alcance y los criterios de evaluación del trabajo. El capítulo 3 revisa el estado del arte y las tecnologías en las que se apoya la propuesta. El capítulo 4 describe el diseño y la arquitectura de la solución, mientras que el capítulo 5 detalla la implementación de Model2Blockly. El capítulo 6 expone el ejemplo conductor y la evaluación de la herramienta. Por último, el capítulo 7 recoge las conclusiones, las limitaciones identificadas y las posibles líneas de trabajo futuro.

Capítulo 2

Objetivos y alcance

Este capítulo delimita el propósito y la extensión del trabajo. En primer lugar, se formula el objetivo general y se concreta en una serie de objetivos específicos. A continuación, se describe el alcance de la solución y se señalan los aspectos que quedan fuera de él. Finalmente, se establecen los criterios que se utilizarán para evaluar la viabilidad y la reutilización del enfoque propuesto.

2.1 Objetivo general

El objetivo general de este Trabajo Fin de Máster es diseñar e implementar un entorno para crear lenguajes de dominio específico con sintaxis basada en bloques. El enfoque propuesto consiste en describir el dominio a alto nivel mediante un metamodelo Ecore anotado o mediante una descripción textual `.m2b`, transformar esa entrada en un modelo intermedio EMF y generar a partir de esa representación un editor Blockly ejecutable. La ruta textual `.m2b` se entiende como un DSL de definición de lenguajes: la usa el diseñador del lenguaje para especificar el editor, mientras que el usuario final interactúa con el editor Blockly generado.

Este objetivo combina dos enfoques. Por un lado, Blockly ofrece la infraestructura para crear editores visuales, pero el desarrollador debe definir los bloques, campos, conexiones y generadores [3]. Por otro lado, la ingeniería dirigida por modelos usa modelos como artefactos centrales para especificar y generar software [5]. El TFM conecta ambas ideas para reducir el trabajo manual necesario en cada nuevo lenguaje.

2.2 Objetivos específicos

Para alcanzar el objetivo general se han definido los siguientes objetivos específicos:

OE1. Analizar la estructura de los lenguajes basados en bloques. Identificar qué necesita un editor de bloques para representar un dominio: tipos de bloques, campos, categorías, conexiones, restricciones, serialización y, cuando sea necesario, generación de código. Este objetivo se justifica por la variedad de diseños que existen en la programación por bloques [1].

OE2. Definir una ruta de generación basada en metamodelos Ecore. Permitir que un metamodelo de dominio actúe como fuente principal de generación. Las clases se mapean a tipos de bloque, los atributos a campos, las referencias de contención a entradas estructurales y las referencias no contenedoras a mecanismos de selección o enlace entre elementos. Las cardinalidades se traducen en comprobaciones sobre cuántos elementos son obligatorios o están permitidos. Esta ruta representa el enfoque metamodelo-céntrico del trabajo: generar código Blockly a partir de un metamodelo, posiblemente anotado con información adicional sobre su sintaxis visual basada en bloques.

OE3. Proporcionar una ruta textual .m2b para definir lenguajes de bloques. Como complemento a la ruta Ecore, se define un DSL textual implementado con Xtext para describir dominios de forma compacta. Su función es ofrecer una notación de autoría para diseñadores de lenguajes, capaz de declarar categorías, clases, atributos, contenciones, referencias, validaciones y metadatos de interfaz, y normalizar después esa información hacia el mismo modelo intermedio. Xtext es adecuado para esta tarea porque genera infraestructura de parsing, modelo abstracto basado en EMF y soporte de edición a partir de una gramática [10, 11].

OE4. Diseñar un modelo intermedio independiente de la salida. Separar la transformación del dominio de la generación de Blockly. Para ello, la ruta Ecore y la ruta textual .m2b se normalizan en una representación común EditorSpec. Así, el generador no depende de los detalles de lectura de Ecore ni de la sintaxis textual y se pueden añadir nuevas salidas con menos cambios.

OE5. Generar editores Blockly ejecutables. Implementar un generador que produzca los ficheros necesarios para abrir un editor en el navegador: definición de bloques, paleta, generadores, validaciones y página HTML. El editor generado permite crear programas o modelos

usando la sintaxis visual de bloques. El estado del workspace puede guardarse en JSON y cargarse posteriormente en el editor. También es posible exportar e importar la representación JSON del modelo. Además, el contenido puede exportarse como XMI de instancia del dominio para su procesamiento con herramientas compatibles con EMF, aunque actualmente este XMI no puede cargarse de nuevo en el editor. La generación también debe producir artefactos revisables, como el XMI intermedio, un modelo de ejemplo y un informe de generación.

OE6. Incorporar mecanismos genéricos de validación, referencias y personalización.

Un editor generado resulta más útil si no se limita a mostrar bloques aislados. Por ello, el entorno incorpora comprobaciones para detectar modelos incompletos, referencias entre elementos del modelo, opciones de presentación y anotaciones que permiten adaptar la apariencia del editor al dominio. Además, las reglas de validación se materializan como una vista Blockly separada, de modo que puedan inspeccionarse visualmente y, en los casos soportados, aplicarse de vuelta a la fuente.

OE7. Evaluar la viabilidad mediante un ejemplo conductor y pruebas. La solución se evalúa con AppMaker descrito por las dos rutas: como metamodelo Ecore con anotaciones y como fichero textual .m2b. Este caso permite analizar si ambas entradas pueden converger en el mismo modelo intermedio y generar un editor Blockly operativo con artefactos revisables. Las pruebas automáticas cubren el modelo intermedio, los adaptadores y los generadores.

2.3 Alcance de la solución

El alcance de la solución se organiza alrededor de una cadena generativa: describir el dominio, transformarlo en una representación intermedia y generar un editor Blockly ejecutable. La entrada puede ser un metamodelo Ecore anotado o un fichero textual .m2b. En ambos casos, el sistema construye una especificación intermedia y genera una implementación HTML/JavaScript que puede abrirse en un navegador.

La Tabla 2.1 resume las dimensiones consideradas en la solución. Estas dimensiones no se organizan como una lista aislada de funciones, sino como los elementos necesarios para sostener el argumento central del trabajo: que un dominio descrito a alto nivel puede transformarse de forma sistemática en una sintaxis visual basada en bloques.

Dimensión	Papel en la solución
Entrada Ecore	Uso de clases, atributos, referencias, herencia, cardinalidades y anotaciones como información de partida para construir el editor visual.
Ruta textual .m2b	Sintaxis Xtext para describir dominios, categorías, clases, atributos, referencias, contenciones, validaciones y plantillas de código con una notación más compacta, normalizable al mismo modelo intermedio.
Modelo intermedio	Representación común de bloques, campos, categorías, referencias, entradas de valor, validaciones y metadatos de interfaz, independiente de la ruta de entrada.
Generación Blockly	Producción de bloques, toolbox, generadores, validaciones y páginas HTML ejecutables a partir de la representación intermedia.
Integración Eclipse	Plugins Eclipse/Xtext para editar ficheros .m2b, ejecutar la generación desde el menú contextual y publicar la herramienta como feature instalable desde un update site p2.
Artefactos trazables	Generación del XMI intermedio, informe de generación, modelo de ejemplo, workspace visual de validaciones y documentación breve de la salida producida.
Validaciones	Derivación de comprobaciones sobre campos obligatorios, mínimos y máximos de elementos, unicidad, referencias obligatorias, expresiones simples y restricciones acotadas de orden.
Referencias	Representación de relaciones de contención y no contención, incluyendo listas desplegadas dinámicas para seleccionar elementos del modelo.
Personalización	Uso de colores, etiquetas, categorías, tooltips, widgets y metadatos de interfaz para adaptar la sintaxis visual al dominio.
Evaluación	Uso de AppMaker por la ruta Ecore y por la ruta textual .m2b, junto con pruebas automáticas para aportar evidencia sobre adaptadores, generadores y modelo intermedio.

Tabla 2.1: Dimensiones incluidas en el alcance de la solución

2.4 Límites del trabajo

Los límites del trabajo se derivan de su orientación generativa. El objetivo no es construir una plataforma comercial completa ni un entorno final para un único dominio. El objetivo es mostrar una cadena reutilizable para crear lenguajes de dominio específico con sintaxis basada en bloques.

Por lo tanto, quedan fuera del alcance principal:

- la importación genérica de modelos XMI existentes a un workspace Blockly;
- la integración completa de lenguajes de restricciones complejos como OCL;
- la sincronización automática de referencias bidireccionales complejas;
- la creación de interfaces de usuario finales altamente especializadas para cada dominio;
- la generación de código con análisis semántico avanzado o formateadores específicos de lenguajes de propósito general.

Algunos de estos elementos se exploran parcialmente, como la exportación a JSON o XMI, la sincronización de referencias opuestas no contenidas, la traducción de un subconjunto sencillo de invariantes OCL, la edición visual de reglas de validación y la generación de código mediante plantillas. Se consideran extensiones del objetivo principal y no equivalen a un entorno completo de modelado, validación y ejecución. Por lo tanto, se retoman explícitamente como líneas de trabajo futuro en el Capítulo 7, donde se discuten como ampliaciones posibles de la herramienta y no como requisitos de la evaluación actual.

2.5 Criterios de evaluación

La evaluación del trabajo se apoya en los siguientes criterios, orientados a valorar si el enfoque propuesto es técnicamente viable y reutilizable:

1. permitir definir distintos dominios mediante metamodelos Ecore con estructuras y características diferentes;

2. generar automáticamente editores Blockly funcionales a partir de dichos metamodelos, sin introducir modificaciones específicas en el generador para cada dominio;
3. permitir complementar la generación con información de presentación, como etiquetas, colores o categorías;
4. ofrecer una sintaxis textual con Xtext para definir lenguajes de bloques sin editar directamente Ecore en casos sencillos o ejemplos;
5. generar código HTML/JavaScript que pueda ejecutarse en un navegador;
6. producir artefactos de trazabilidad, como XMI intermedio, informe de generación y ficheros de validación revisables;
7. demostrar la reutilización de la arquitectura con un dominio representativo descrito por la ruta Ecore y por la ruta textual .m2b, normalizadas al mismo modelo intermedio;
8. disponer de pruebas, verificaciones y un mecanismo de instalación que reduzcan el riesgo de que la solución funcione únicamente para un caso preparado manualmente.

Estos criterios responden a la motivación inicial del trabajo: evitar que cada lenguaje basado en bloques requiera escribir de nuevo la infraestructura Blockly de bajo nivel. Si el dominio puede describirse como metamodelo Ecore o como modelo textual .m2b, el sistema lo transforma a EditorSpec y genera un editor operativo, entonces el enfoque aporta evidencia frente a la programación manual de cada editor.

Capítulo 3

Estado del arte y tecnologías

Este capítulo presenta los fundamentos conceptuales y tecnológicos de la propuesta. En primer lugar, se revisan la programación basada en bloques, Blockly y los principios de los lenguajes de dominio específico y de la ingeniería dirigida por modelos. A continuación, se describen EMF, Ecore y Xtext, tecnologías utilizadas en el desarrollo de Model2Blockly. Después se analizan los trabajos relacionados sobre DSLs basados en bloques y se compara la propuesta con ellos para precisar su posicionamiento. Finalmente, se explica la relación entre las tecnologías seleccionadas dentro de la cadena de generación planteada.

3.1 Programación basada en bloques

La programación basada en bloques sustituye parte de la escritura textual por piezas visuales. Estas piezas representan instrucciones, expresiones o estructuras de control. El entorno define cómo se pueden conectar, por lo que se evitan muchas combinaciones sintácticamente incorrectas. Además, el usuario puede ver qué construcciones están disponibles. Por esta razón, los entornos de bloques se usan con frecuencia para introducir la programación a usuarios noveles [1, 12]. Herramientas de análisis como Dr. Scratch muestran además que los programas de bloques pueden inspeccionarse y evaluarse automáticamente, lo que refuerza su interés como representación estructurada y no solo como interfaz gráfica [13].

Los bloques no son solo una forma gráfica de mostrar código. La forma en que el usuario representa y manipula las construcciones del lenguaje influye en su forma de programar [2]. Por eso, al diseñar un lenguaje de bloques hay que decidir qué conceptos serán bloques, qué

parámetros serán campos, cómo se organizará la paleta, qué conexiones serán válidas y qué restricciones se mostrarán al usuario.

La Figura 3.1 muestra un ejemplo de esta forma de programación en Music Maker, una aplicación web desarrollada en el codelab oficial de Blockly [14]. En el modo de edición, el usuario construye un programa encajando un bloque que reproduce la nota D4 dentro de una estructura que repite la acción tres veces. La forma de las piezas expresa la relación estructural entre las instrucciones, mientras que los campos permiten configurar la repetición y la nota sin escribir código textual. El modo de reproducción presenta los botones de la aplicación sobre los que se ejecutan los programas musicales definidos mediante bloques.

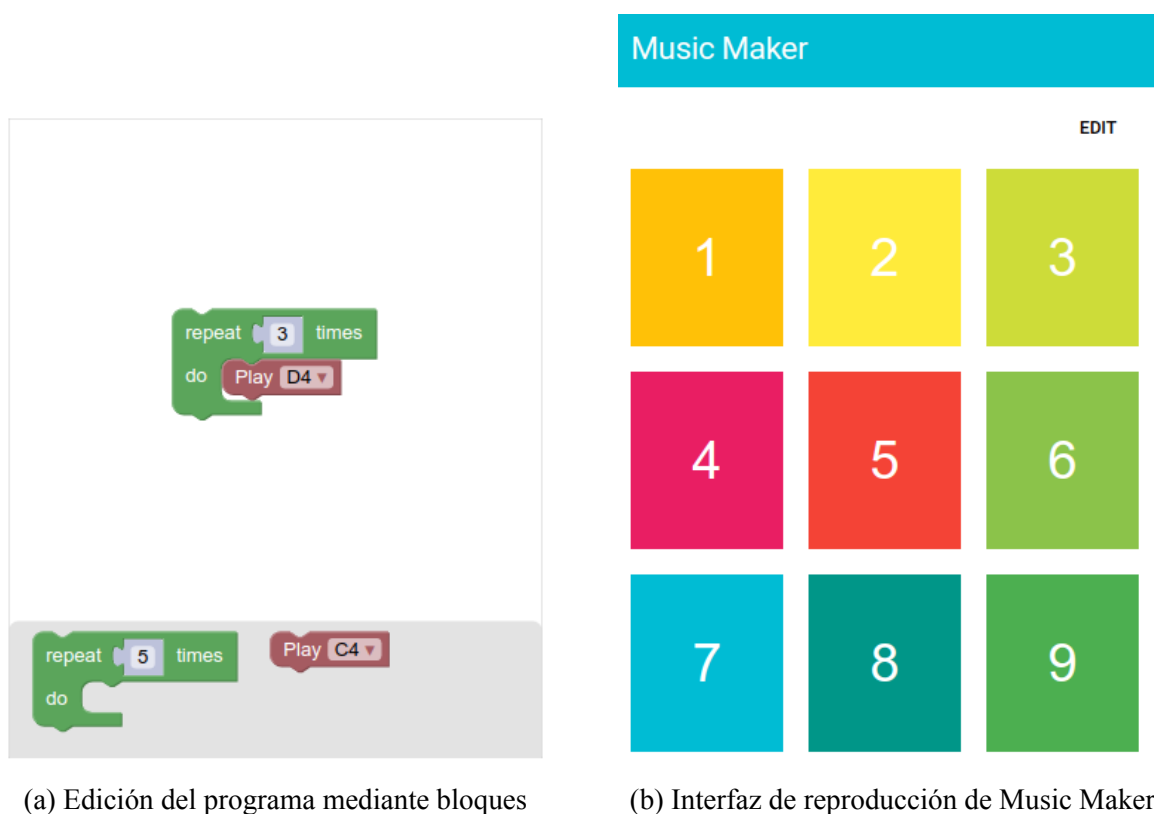


Figura 3.1: Edición y reproducción de un programa basado en bloques en Music Maker. Fuente: codelab oficial de Blockly [14]

En este TFM, la programación por bloques se entiende como una sintaxis concreta visual, es decir, como la forma visible con la que el usuario manipula un lenguaje de dominio específico. El usuario final no necesita conocer la implementación JavaScript del editor ni la estructura interna del metamodelo; interactúa con un conjunto de bloques que representan

conceptos del dominio. Esta idea conecta directamente con la motivación del trabajo: crear lenguajes de bloques para diferentes dominios sin programar manualmente cada editor desde cero.

3.2 Blockly

Blockly es una biblioteca web de código abierto para construir editores basados en bloques. La documentación oficial describe Blockly como una biblioteca que permite integrar un editor de código visual en una aplicación web; el desarrollador define los bloques y la semántica asociada, mientras que la biblioteca proporciona la infraestructura de interacción visual, arrastre, encaje, toolbox y serialización [3, 4].

La biblioteca permite definir bloques con campos, entradas de valor, entradas de sentencias, conexiones previas y posteriores, colores, tooltips y generadores de código. También permite organizar los bloques en categorías de paleta o *toolbox*, y serializar el estado del área de trabajo o *workspace* [15]. Estas capacidades hacen que Blockly sea una base adecuada para construir lenguajes visuales específicos, pero no eliminan la necesidad de definir manualmente la configuración de cada dominio.

Antes de definir elementos específicos del dominio, el desarrollador debe cargar las dependencias, proporcionar un contenedor HTML e inicializar el workspace. El Listado 3.1 resume esta infraestructura mínima siguiendo el codelab oficial [14].

```
<script src="https://unpkg.com/blockly/blockly_compressed.js"></script>
<script src="https://unpkg.com/blockly/blocks_compressed.js"></script>
<script src="https://unpkg.com/blockly/javascript_compressed.js"></script>
<script src="https://unpkg.com/blockly/msg/en.js"></script>

<div id="blocklyDiv"></div>

Blockly.inject("blocklyDiv", {"toolbox": toolbox});
```

Listado 3.1: Infraestructura mínima para crear un workspace Blockly

El Listado 3.2 muestra un ejemplo reducido de esa configuración, tomado del codelab oficial Music Maker [14]. El primer objeto define un bloque con un campo desplegable para seleccionar una nota y con conexiones anterior y posterior. El segundo determina qué tipos de bloque aparecen en la toolbox. Aunque se habla habitualmente de definiciones JSON, en este caso se expresan como objetos JavaScript cuyos valores son serializables como JSON.

```
Blockly.common.defineBlocksWithJsonArray([
  {
    "type": "play_sound",
    "message0": "Play %1",
    "args0": [
      {
        "type": "field_dropdown",
        "name": "VALUE",
        "options": [
          ["C4", "sounds/c4.m4a"],
          ["D4", "sounds/d4.m4a"]
        ]
      }
    ],
    "previousStatement": null,
    "nextStatement": null,
    "colour": 355
  }
]);

const toolbox = {
  "kind": "flyoutToolbox",
  "contents": [
    { "kind": "block", "type": "controls_repeat_ext" },
    { "kind": "block", "type": "play_sound" }
  ]
};
```

Listado 3.2: Definición de un bloque y configuración mínima de una toolbox en Blockly

La Figura 3.2 muestra el editor resultante: la toolbox ofrece el bloque de repetición y el bloque musical, que pueden arrastrarse al workspace para construir el programa.

Aunque el editor resultante solo ofrece un bucle y una operación musical, su construcción exige gestionar dependencias, preparar el contenedor, configurar el workspace y declarar manualmente la estructura, los campos, las conexiones y la toolbox. Todavía habría que añadir el generador de código y la lógica de ejecución. Este esfuerzo crece con el número de conceptos y restricciones del dominio y se repite para cada editor. La Figura 3.1, presentada en la sección anterior, muestra cómo el usuario combina posteriormente esos bloques para definir el comportamiento de Music Maker.

El problema que aborda este TFM aparece precisamente en ese punto. Si se desea crear un editor Blockly para un dominio de planificación, otro para un dominio de storytelling y otro para un dominio de robótica, gran parte del trabajo se repite: declarar bloques, campos, categorías, conexiones, validaciones y generadores. El enfoque propuesto consiste en generar esos elementos a partir de una descripción abstracta del dominio. Así, Blockly se utiliza como plataforma de ejecución visual, pero la especificación del lenguaje se mantiene en un nivel

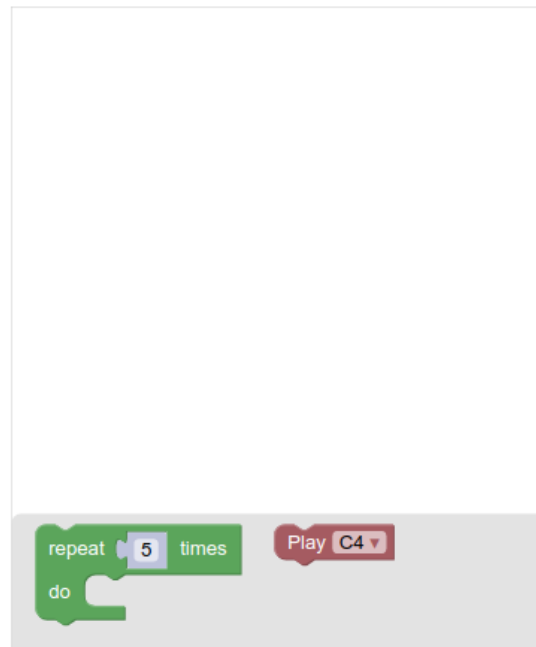


Figura 3.2: Workspace y toolbox obtenidos tras registrar el bloque play_sound. Fuente: codelab oficial de Blockly [14]

de metamodelo o DSL.

3.3 Lenguajes de dominio específico e ingeniería dirigida por modelos

Un lenguaje de dominio específico ofrece conceptos y notaciones adaptados a una familia concreta de problemas. A diferencia de un lenguaje de propósito general, un DSL permite usar términos propios del dominio. En enfoques basados en metamodelado, un DSL suele tener una sintaxis abstracta, definida mediante un metamodelo, y una o varias sintaxis concretas con las que trabaja el usuario [16, 17].

La ingeniería dirigida por modelos ofrece un marco para esta separación. En MDE, los modelos son artefactos principales del desarrollo. Se usan para especificar, diseñar, analizar, transformar y generar software [5]. En la industria, algunas motivaciones para usar MDE son reducir tiempo de desarrollo, reutilizar soluciones y mejorar la calidad [5-7]. Estas ideas encajan con este TFM: usar un generador para producir editores Blockly en vez de programar cada editor desde cero.

En el ámbito del modelado, existen también estándares generales como UML y OCL. UML define una notación gráfica estandarizada para visualizar, especificar, construir y documentar sistemas software, mientras que OCL proporciona un lenguaje formal para expresar restricciones sobre modelos [18, 19]. Estos estándares son relevantes como contexto, pero el presente TFM no propone un perfil UML ni una integración completa de OCL. La decisión tomada es más concreta: usar un metamodelo como descripción abstracta del dominio, enriquecerlo con información sobre la sintaxis visual y derivar a partir de ella una sintaxis concreta basada en bloques. En este trabajo, esta función se implementa mediante Ecore, tecnología que se presenta en la Sección 3.4. El DSL textual del proyecto se entiende como una segunda ruta de autoría para diseñadores de lenguajes: permite describir la estructura del editor Blockly de forma más compacta y normalizarla al mismo modelo intermedio.

La experiencia acumulada en lenguajes específicos de dominio muestra además que las herramientas de modelado pueden elevar el nivel de abstracción al expresar soluciones directamente con conceptos del dominio y generar artefactos a partir de esos modelos [20]. Esta idea coincide con la motivación de Model2Blockly: que el usuario defina los conceptos del dominio y que la infraestructura genere el editor visual correspondiente.

En este proyecto, la sintaxis basada en bloques se considera una sintaxis concreta visual del dominio. El metamodelo Ecore describe qué conceptos existen y cómo se relacionan; las anotaciones aportan información de presentación que Ecore no expresa de forma nativa; la

ruta textual .m2b puede escribir la definición del lenguaje de bloques en un formato más breve. El generador produce la configuración Blockly que implementa la sintaxis visual usada por el usuario final.

3.4 EMF y Ecore

Eclipse Modeling Framework (EMF) es una infraestructura de modelado para definir modelos estructurados y generar código a partir de ellos. Ecore es el metamodelo central de EMF y permite representar paquetes, clases, atributos, referencias, tipos de datos, herencia y cardinalidades [8, 9]. En la práctica, Ecore proporciona una forma compacta y ampliamente utilizada de describir la estructura abstracta de un dominio.

En este TFM, Ecore cumple tres funciones. Primero, describe la estructura del dominio: las clases son conceptos, los atributos son propiedades y las referencias son relaciones. Segundo, permite obtener información para la sintaxis de bloques: una clase puede ser un tipo de bloque, un atributo puede ser un campo y una referencia de contención puede ser una entrada para bloques hijos. Tercero, Ecore permite añadir anotaciones de presentación, como etiquetas, colores, categorías o tooltips.

La Figura 3.3 representa el fragmento del metamodelo Ecore relevante para estas funciones. Se omiten algunos tipos intermedios, como `ENamedElement` y `ETypedElement`, para mantener legible el diagrama.

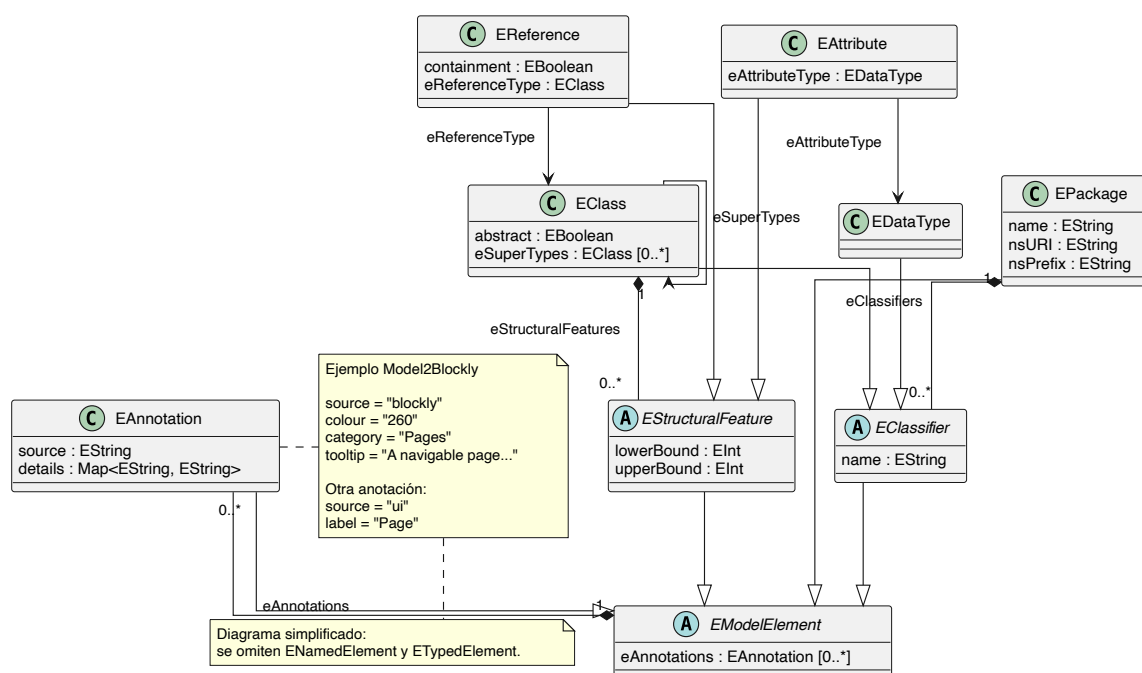


Figura 3.3: Fragmento simplificado del metamodelo Ecore y uso de anotaciones en Model2Blockly

Un EPackage agrupa los clasificadores del dominio. Entre ellos, una EClass representa un concepto y puede contener características estructurales: los EAttribute almacenan valores tipados y las EReference relacionan instancias de clases, con la posibilidad de marcar una relación como contención. Los límites inferior y superior expresan la cardinalidad de cada característica. Estos elementos heredan, directa o indirectamente, de EModelElement, que permite asociar instancias de EAnnotation. Cada anotación identifica su propósito mediante source y almacena pares clave-valor en details; Model2Blockly utiliza este mecanismo para incorporar información de sintaxis visual sin alterar la estructura abstracta del dominio. En el ejemplo de la figura, la anotación con source="blockly" define colour, category y tooltip, mientras que una anotación separada con source="ui" proporciona la etiqueta visible mediante label.

Esta capacidad de separar estructura e información de presentación es esencial para el trabajo. No todos los detalles de un editor Blockly existen de forma natural en un metamodelo Ecore: por ejemplo, el color de un bloque o la organización de la toolbox son decisiones de sintaxis concreta. Por ello, el proyecto permite enriquecer el metamodelo con anotaciones

específicas, sin perder la posibilidad de generar un editor básico a partir de la estructura Ecore.

3.5 Xtext

Xtext es un framework de Eclipse para el desarrollo de lenguajes textuales específicos de dominio. A partir de una gramática, Xtext genera infraestructura de análisis sintáctico, un modelo abstracto basado en EMF y soporte de edición como resaltado de sintaxis, navegación, autocompletado y marcadores de error [10, 11, 21]. Estas características lo hacen adecuado para construir una ruta textual alternativa a la edición directa de metamodelos Ecore.

Conviene distinguir dos momentos en el funcionamiento de Xtext. En primer lugar, durante la construcción de la herramienta, el desarrollador escribe una gramática que describe las palabras clave, las reglas y las referencias del lenguaje. Xtext procesa esa gramática y genera el analizador, la infraestructura EMF y los servicios del editor. En segundo lugar, durante el uso de la herramienta, ese analizador lee cada fichero escrito en el nuevo lenguaje y construye una instancia de su modelo abstracto. Por lo tanto, Xtext no genera el editor Blockly directamente: genera la infraestructura que permite comprender una especificación textual, que después Model2Blockly transforma.

La Figura 3.4 resume este mecanismo. La gramática determina qué textos son válidos y cómo se relacionan con los conceptos del modelo. A partir de ella se obtienen el parser y los servicios de edición. Cuando el autor escribe una especificación, el parser construye un grafo de objetos EMF; el enlazado resuelve nombres que hacen referencia a otros elementos y el validador comunica los problemas detectados. Una transformación posterior puede entonces consumir ese modelo sin volver a interpretar el texto.

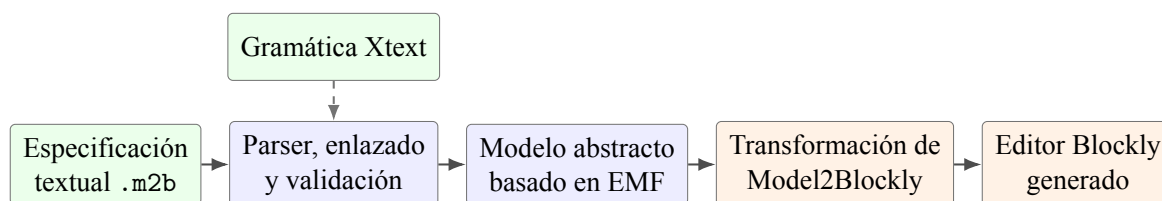


Figura 3.4: Funcionamiento general de Xtext y su aplicación en Model2Blockly

En este proyecto, Xtext se usa para definir el lenguaje textual .m2b. El Listado 3.3 ilustra

su nivel de abstracción: el autor declara conceptos del dominio, propiedades y relaciones, en lugar de programar directamente llamadas a la API JavaScript de Blockly.

```
domain Appmaker
category Pages label "Pages" colour 260

abstract class Component { }
class Page category Pages {
  attribute title : string default "Home" required
  contains Component components [1..40]
}
```

Listado 3.3: Ejemplo breve de una definición textual .m2b

La declaración `class Page` representa un concepto que podrá convertirse en un tipo de bloque; `title` aporta una propiedad editable y `components` expresa una relación jerárquica con cardinalidad. Xtext comprueba que la entrada respeta la gramática y que nombres como `Pages` y `Component` se pueden resolver. El resultado es un modelo EMF que la cadena de `Model2Blockly` puede transformar posteriormente. Este mecanismo concreto se desarrolla desde el punto de vista arquitectónico en la Sección 4.5 y desde el punto de vista de implementación en la Sección 5.5.

La ruta Xtext no sustituye, por lo tanto, a la ruta Ecore; la complementa. Ecore resulta adecuado cuando ya existe un metamodelo EMF o cuando se quiere mantener la definición del dominio dentro de dicho ecosistema. La ruta .m2b resulta útil cuando se desea reunir en un fichero legible la estructura del dominio y las decisiones de sintaxis concreta, como etiquetas, colores, widgets, organización de la toolbox y plantillas de código. La convergencia en EditorSpec evita mantener dos generadores diferentes.

3.6 Xtend

Xtend es un lenguaje de programación estáticamente tipado del ecosistema Xtext. Su sintaxis parte de Java, pero añade construcciones más concisas, como inferencia de tipos, métodos de extensión, expresiones lambda, expresiones `switch` y plantillas de texto. El compilador de Xtend traduce los ficheros .xtend a código fuente Java, por lo que el código resultante se ejecuta sobre la JVM y puede utilizar directamente clases y bibliotecas Java [22].

Para la generación de código resultan especialmente útiles las expresiones de plantilla.

Una plantilla se delimita mediante triples comillas simples y permite insertar expresiones entre los caracteres « y ». Xtend evalúa esas expresiones, incorpora su representación textual y aplica reglas de indentación que ayudan a mantener legibles tanto la plantilla como la salida generada [22]. El Listado 3.4 muestra un fragmento simplificado, basado en el generador del proyecto.

```
def String generateBlocksJs(EditorSpec spec) '''
    // Block definitions for domain "«spec.domainName»".
    window.BLOCKLY_BLOCKS = «generateBlocksArray(spec)»;
'''
```

Listado 3.4: Ejemplo de una plantilla Xtend usada para generar JavaScript

En el ejemplo, `def` declara un método y la plantilla constituye su resultado. Las expresiones insertadas obtienen el nombre del dominio y la lista de bloques calculada por otro método. El mismo mecanismo permite combinar texto HTML o JavaScript con recorridos, condiciones y datos procedentes de un modelo EMF, sin construir la salida mediante numerosas concatenaciones de cadenas.

Dentro de `Model2Blockly`, Xtend no define el lenguaje `.m2b` ni forma parte del editor generado. Se utiliza como tecnología de implementación en dos puntos: `Model2BlocklyGenerator` integra la generación con el flujo de Xtend, y `BlocklyCodeGenerator` contiene las transformaciones y plantillas que producen bloques, toolbox, validaciones y páginas HTML. Gracias a la interoperabilidad con Java, estas clases invocan los adaptadores, serializadores y utilidades Java del proyecto, y también pueden ser invocadas desde las entradas independientes escritas en Java. Los detalles de esta implementación se presentan en la Sección 5.9.

3.7 Trabajos relacionados sobre DSLs basados en bloques

Además de las tecnologías utilizadas, existen trabajos que tratan de reducir el esfuerzo de construir lenguajes basados en bloques. Pasternak, Fenichel y Marshall describen recomendaciones prácticas para crear lenguajes con Blockly: decidir el estilo de los bloques, seleccionar qué conceptos deben ser visibles y mantener la complejidad de la API alejada del usuario final [23]. Este trabajo es relevante porque muestra que Blockly es flexible, pero también que diseñar un lenguaje de bloques implica decisiones de ingeniería que no desaparecen al usar la biblioteca.

Merino y van der Storm plantean el problema desde la perspectiva de los *language workbenches*: la creación de lenguajes de bloques sigue siendo a menudo una tarea de bajo nivel y debería poder abordarse con metalenguajes y herramientas específicas [24]. En una línea posterior, Kogi deriva entornos de bloques desde gramáticas libres de contexto y define una estructura intermedia para describir el entorno generado [25]. Blocklybench propone un entorno basado en bloques para describir otros entornos de bloques, incluyendo aspectos como disposición, colores, conexiones y generación de código [26].

Model2Blockly se sitúa cerca de estos trabajos porque también busca elevar el nivel de abstracción al crear editores de bloques. La diferencia principal está en la fuente de información: en lugar de partir de gramáticas textuales libres de contexto o de un metaeditor de bloques, esta propuesta admite metamodelos Ecore anotados y modelos textuales .m2b. Ambas rutas convergen en un modelo intermedio definido explícitamente como metamodelo EMF, para mantener una cadena MDE entre la transformación de entrada y el generador HTML/JavaScript.

3.8 Comparación y posicionamiento de la propuesta

Las tecnologías anteriores cubren partes distintas del problema, pero ninguna resuelve por sí sola el objetivo de este TFM. Blockly ofrece la infraestructura de bloques, pero exige configurar cada dominio a mano. EMF/Ecore describe la estructura abstracta de un dominio, pero no genera un editor de bloques. Xtext ayuda a crear lenguajes textuales, pero su salida natural no es un editor Blockly. UML y OCL son referencias importantes para modelado y restricciones, pero este trabajo se centra en una cadena generativa más concreta.

La Tabla 3.1 resume esta comparación y sitúa el papel de Model2Blockly dentro del contexto revisado. La propuesta no pretende reemplazar estas tecnologías, sino conectarlas mediante una representación intermedia que permita derivar una sintaxis concreta visual desde una descripción de dominio.

Enfoque	Aportación principal	Limitación respecto al objetivo del TFM
Desarrollo manual con Blockly	Permite construir editores visuales ricos y ejecutables en la web.	Requiere codificar bloques, toolbox, conexiones, validaciones y generadores para cada dominio.
Metamodelado con Ecore	Define de forma precisa la estructura abstracta del dominio.	No produce automáticamente una sintaxis concreta basada en bloques.
Sintaxis textual con Xtext	Genera infraestructura de lenguaje textual, validación y edición. En este TFM se usa para la ruta .m2b.	No convierte por sí sola esa definición en un editor visual Blockly.
Restricciones generales con OCL	Expresa propiedades formales sobre modelos.	Supera el alcance de las validaciones estructurales generadas en este trabajo.
Kogi y Blocklybench	Generan o describen entornos de bloques desde artefactos de lenguaje o metaentornos específicos.	No toman como fuente principal metamodelos Ecore anotados ni usan un EditorSpec EMF como contrato XMI explícito para la generación.
Model2Blockly	Conecta metamodelos Ecore anotados y modelos .m2b con editores Blockly mediante transformación, modelo intermedio y generación.	Su validación y evaluación se limitan al conjunto de casos y reglas estructurales estudiados.

Tabla 3.1: Posicionamiento de Model2Blockly frente a enfoques relacionados

3.9 Relación entre las tecnologías

La relación entre las tecnologías utilizadas puede resumirse como una cadena de transformación. EMF/Ecore permite definir la estructura abstracta del dominio y las anotaciones añaden metadatos de sintaxis concreta. Xtext ofrece la ruta textual .m2b para escribir definiciones compactas de lenguajes de bloques. Blockly proporciona la infraestructura web para ejecutar la sintaxis visual basada en bloques. El generador desarrollado en Java y Xtend conecta estos elementos mediante un modelo intermedio EMF independiente de la salida final; Xtend se emplea principalmente para expresar las plantillas textuales de la salida.

La Tabla 3.2 resume el papel de cada tecnología dentro del proyecto.

Esta combinación da soporte técnico al enfoque del proyecto: definir lenguajes de dominio específico a un nivel abstracto y generar automáticamente una sintaxis concreta basada

Tecnología	Papel en el TFM
Blockly	Plataforma web para representar bloques, toolbox, workspace, conexiones y generación asociada.
EMF/Ecore	Definición estructural del dominio mediante metamodelos, clases, atributos, referencias y cardinalidades.
Xtext	Ruta textual .m2b para definir lenguajes de bloques y generar modelos EMF a partir de descripciones compactas.
Java	Implementación de adaptadores, validadores, serialización del modelo intermedio y utilidades de ejecución.
Xtend	Implementación del punto de entrada de generación de Xtext y de las plantillas que producen los artefactos HTML/JavaScript.
HTML/JavaScript	Tecnología de salida principal para ejecutar el editor Blockly generado en el navegador.

Tabla 3.2: Papel de las tecnologías principales en el proyecto

en bloques. El valor de la solución no reside en usar Blockly de forma aislada, sino en integrarlo con metamodelado Ecore, transformación de modelos y generación para crear editores reutilizables en distintos dominios.

Capítulo 4

Diseño y arquitectura de la solución

La arquitectura de Model2Blockly considera que un editor Blockly es una sintaxis visual de un lenguaje de dominio específico. Por eso, el sistema debe partir de una descripción abstracta del dominio y producir una implementación web ejecutable.

El capítulo se organiza de la siguiente manera. En primer lugar, se presenta el enfoque arquitectónico y el flujo general de generación. A continuación, se describe el modelo intermedio que conecta las entradas con los generadores. Después se justifican las rutas basadas en Ecore y en el DSL textual `.m2b`, así como la generación de los editores Blockly. Finalmente, se exponen los mecanismos de validación, referencias y personalización, y se relacionan las decisiones arquitectónicas con los objetivos del trabajo.

4.1 Enfoque general

La arquitectura del sistema sigue un enfoque de ingeniería dirigida por modelos. En lugar de definir manualmente cada bloque en JavaScript, el diseñador describe el dominio mediante un metamodelo Ecore anotado o mediante una definición textual `.m2b`. A partir de esa descripción, Model2Blockly deriva la estructura y parte del comportamiento del editor. Esta organización aplica los principios de MDE al diseño concreto de la solución: los modelos constituyen la entrada principal del proceso y se utilizan para generar software [5].

En ambas rutas se mantiene la separación habitual entre sintaxis abstracta y sintaxis concreta de un DSL [16]. Ecore o el lenguaje `.m2b` describen los conceptos y relaciones del dominio, mientras que Blockly proporciona la notación visual con la que el usuario final

crea y edita instancias. Por lo tanto, el lenguaje textual de Xtext no sustituye al editor visual: es una forma compacta de especificar qué lenguaje de bloques debe generarse.

El diseño se apoya en tres principios. El primero es separar la interpretación del dominio de la generación del editor. El segundo es hacer que las dos rutas de entrada produzcan un modelo EditorSpec conforme al mismo metamodelo intermedio. Este modelo constituye el contrato arquitectónico compartido antes de generar HTML y JavaScript. El tercero es derivar no solo la apariencia de los bloques, sino también parte de su comportamiento: conexiones, validaciones, referencias, exportación y metadatos de interfaz.

Estos principios responden al problema identificado en la introducción. Blockly proporciona la infraestructura de interacción visual, pero exige que el desarrollador describa cada bloque, campo, conexión y generador [4]. Model2Blockly desplaza esa descripción a un nivel de dominio: el desarrollador define clases, atributos, contenciones, referencias, anotaciones o construcciones textuales .m2b; la herramienta se encarga de transformar esa información en los artefactos Blockly correspondientes.

4.2 Flujo general de generación

El flujo completo de la solución se organiza en cuatro etapas:

1. definición del dominio mediante un metamodelo Ecore con anotaciones o mediante un fichero textual .m2b;
2. interpretación de la entrada y transformación a un modelo intermedio común;
3. generación de los artefactos Blockly a partir de ese modelo;
4. ejecución del editor generado en el navegador.

La Figura 4.1 muestra las dos rutas de entrada y la cadena común que se ejecuta después de convertirlas al modelo intermedio.

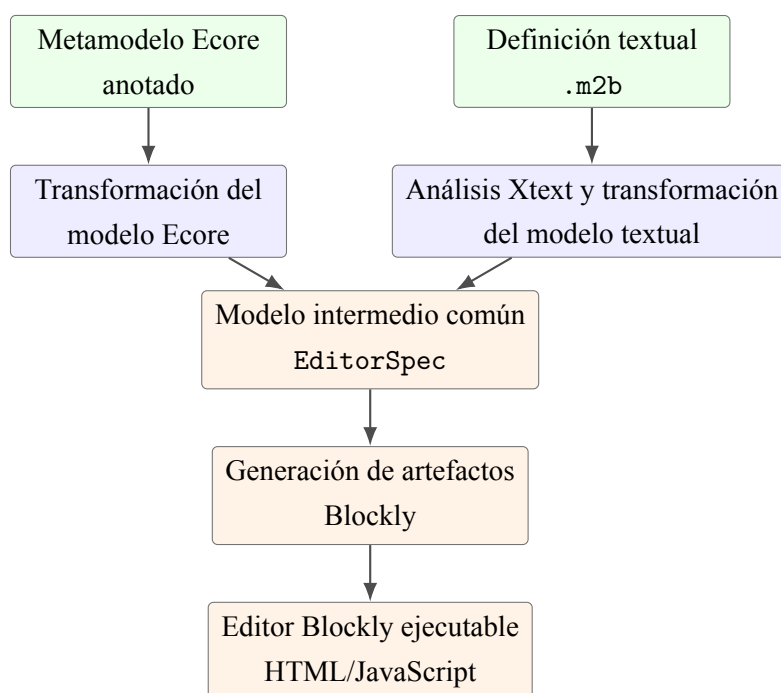


Figura 4.1: Arquitectura general de la generación de editores Blockly

La Figura 4.1 muestra dos recorridos diferentes en su parte superior. En la ruta Ecore se interpreta la estructura del metamodelo y sus anotaciones para construir el modelo intermedio. En la ruta textual, Xtext analiza primero la definición .m2b y crea un modelo estructurado; una segunda transformación lleva ese modelo a EditorSpec. Las dos rutas terminan, por lo tanto, en modelos conformes al mismo metamodelo intermedio.

Por ejemplo, una EClass Page con un atributo title:EString y una declaración textual class Page con attribute title:string se representan de la misma forma después de la conversión: un tipo de bloque Page con un campo de texto title. Desde ese punto se aplica una única generación sobre EditorSpec. Por ello, la solución necesita dos mecanismos de entrada, pero comparte la representación intermedia y la generación de salida. Esta convergencia es la decisión que estructura el resto de la arquitectura.

4.3 Modelo intermedio

Las dos rutas de entrada describen conceptos similares mediante representaciones diferentes. Si la generación trabajara directamente con ambas, tendría que mantener dos interpre-

taciones de clases, propiedades, relaciones y restricciones. La incorporación de una nueva forma de entrada obligaría además a modificar la generación. Para evitar esta dependencia, la arquitectura introduce un modelo intermedio común entre las entradas y la salida.

Este modelo representa el editor que se desea obtener, no una copia completa del artefacto de origen. Su contenido se organiza conceptualmente en tres grupos. El primero describe la estructura general del editor, como las categorías de la toolbox y su configuración. El segundo define los bloques, sus campos y sus formas de conexión. El tercero conserva la información de comportamiento que afecta a la edición, como referencias y reglas de validación. El metamodelo intermedio establece las relaciones válidas entre estos conceptos y cada transformación de entrada produce un modelo conforme a él.

La raíz de esta representación se denomina `EditorSpec`. Por ejemplo, una clase de dominio `Page` se representa como un tipo de bloque y su propiedad `title` como un campo editable. Una relación de composición se representa como una zona para insertar bloques hijos, mientras que una referencia a otro objeto se representa como un mecanismo de selección. El origen puede ser `Ecore` o `.m2b`; después de la transformación, estos conceptos tienen la misma forma para la generación.

La elección de esta representación intermedia permite que cada parte tenga una responsabilidad definida. Las rutas de entrada interpretan el dominio y la generación interpreta únicamente `EditorSpec`. Así se comparte una sola lógica de salida y se pueden modificar o ampliar las entradas sin rediseñar el proceso que produce el editor `Blockly`.

4.4 Ruta basada en Ecore

La ruta `Ecore` está dirigida a dominios que ya se encuentran definidos dentro del ecosistema EMF. Permite reutilizar un metamodelo existente como punto de partida, en lugar de volver a describir manualmente sus conceptos para `Blockly`. `Ecore` aporta la estructura abstracta del dominio mediante clases, atributos, referencias, herencia y cardinalidades [8].

El mapeo básico es sistemático. Cada clase `Ecore` se convierte en un tipo de bloque; cada atributo, en un campo visual; cada referencia de contención, en una entrada estructural para bloques hijos; y cada referencia no contenida, en un campo de referencia dinámica. La herencia se utiliza para mantener la compatibilidad entre tipos de bloque y las cardinalidades permiten derivar comprobaciones sobre elementos obligatorios o cantidades admitidas. Estas reglas de correspondencia preservan en el editor las diferencias estructurales que son

relevantes para crear modelos válidos.

Ecore no describe por sí solo todas las decisiones de sintaxis concreta de un editor Blockly, como el color, la etiqueta visible, la organización de la toolbox o el tipo de control utilizado para editar un valor. Por este motivo, la ruta admite anotaciones que complementan el metamodelo sin alterar su estructura esencial. La separación entre estructura del dominio y metadatos de presentación permite obtener una salida útil por defecto y personalizarla cuando el dominio lo requiere.

4.5 Ruta textual .m2b basada en Xtext

La ruta textual ofrece una alternativa de autoría cuando no se dispone de un metamodelo Ecore previo o se prefiere una notación más compacta. Xtext resulta adecuado porque permite definir una sintaxis textual y obtener a partir de ella una representación estructurada del dominio [10, 11].

Esta ruta no reemplaza a Ecore, sino que responde a una necesidad distinta. El DSL permite reunir en una misma descripción la estructura del dominio y las decisiones básicas de presentación del editor. Xtext se encarga de interpretar el texto y comprobar que respeta la sintaxis definida; posteriormente, esa representación se transforma al mismo modelo intermedio empleado por la ruta Ecore. La generación continúa siendo común y no depende de la forma elegida para describir el dominio.

4.6 Generación de editores Blockly

Una vez construido el modelo intermedio, la generación produce los artefactos necesarios para ejecutar el editor en un navegador. Esta etapa puede entenderse como una transformación de modelo a texto: la especificación del editor se convierte en definiciones de bloques, toolbox, configuración del workspace, reglas de validación y páginas HTML y JavaScript.

La correspondencia se establece por responsabilidad: la organización del modelo intermedio determina la toolbox, la definición de cada tipo determina la forma del bloque y sus conexiones, y las reglas conservadas determinan las ayudas y advertencias disponibles durante la edición. Así, la salida visual se deriva de decisiones expresadas en el modelo y no de una colección de bloques programados manualmente.

La dependencia exclusiva respecto al modelo intermedio evita duplicar la lógica de salida para cada entrada y deja abierta la posibilidad de incorporar otros formatos de origen o nuevos objetivos de generación. Para este trabajo se elige como salida principal un editor HTML/JavaScript autocontenido, porque facilita su ejecución y revisión sin exigir una infraestructura adicional.

4.7 Validaciones, referencias y personalización

Un editor de bloques no debe limitarse a mostrar piezas visuales; también debe ayudar a construir modelos coherentes. Por ello, la arquitectura conserva en el modelo intermedio las restricciones que pueden trasladarse al editor. Entre ellas se encuentran las cardinalidades, la obligatoriedad de valores y algunas relaciones sencillas entre elementos. La decisión de mantenerlas en el modelo intermedio permite aplicar el mismo criterio de validación con independencia de la ruta de entrada.

Estas comprobaciones no constituyen un motor general de restricciones. OCL permite expresar invariantes sobre objetos, propiedades y colecciones [19], mientras que Model2Blockly solo transforma patrones que pueden reconocerse y ejecutarse de manera predecible en el editor generado. Esta decisión mantiene explícito el alcance del sistema: las validaciones ayudan al usuario mediante advertencias, pero no sustituyen a un lenguaje completo de restricciones. La implementación concreta y la evaluación del subconjunto soportado se presentan en los capítulos siguientes.

Las referencias no contenidas se tratan de forma distinta a las contenciones. Una contención define la estructura jerárquica del modelo y por lo tanto se representa como una entrada de sentencias. Una referencia no contenida apunta a otro elemento existente en el workspace, por lo que se representa mediante un desplegable dinámico. El generador calcula los candidatos compatibles teniendo en cuenta el tipo objetivo y sus subtipos. Esta decisión mantiene una diferencia semántica importante del metamodelo: no es lo mismo crear un hijo estructural que seleccionar otro elemento ya presente en el modelo.

La personalización se incorpora como una capa controlada de metadatos. Colores, etiquetas, categorías, tooltips, disposición en línea, widgets de interfaz y plantillas de código no cambian la estructura esencial del dominio, pero sí afectan a la experiencia del usuario final. Model2Blockly permite definir esta información mediante anotaciones Ecore o mediante opciones del DSL textual. De este modo, se evita que el usuario tenga que editar directamente

JavaScript para realizar cambios habituales de presentación.

4.8 Trazabilidad con los objetivos

La arquitectura propuesta mantiene una relación directa con los objetivos específicos definidos para el TFM. La Tabla 4.1 resume esa relación.

Objetivo	Respuesta arquitectónica
OE1	El modelo intermedio identifica explícitamente los elementos que necesita un editor de bloques: tipos, campos, conexiones, categorías, referencias y validaciones.
OE2	La ruta Ecore permite partir de metamodelos y derivar bloques, campos, entradas y reglas.
OE3	La gramática Xtext proporciona la ruta textual .m2b para definir lenguajes de bloques de forma compacta.
OE4	El modelo intermedio común desacopla las entradas de los generadores y permite compartir la misma lógica de salida.
OE5	Los generadores producen editores HTML/JavaScript ejecutables en el navegador.
OE6	Las validaciones, referencias dinámicas, anotaciones de interfaz y plantillas de código se incorporan como capacidades genéricas.
OE7	La separación por transformación de entrada, modelo intermedio y generadores facilita probar cada parte y evaluar la convergencia de las dos rutas sobre AppMaker.

Tabla 4.1: Trazabilidad entre objetivos y arquitectura

En conjunto, la arquitectura define una cadena generativa clara: el diseñador describe un dominio como metamodelo Ecore anotado o como fichero .m2b, el sistema transforma esa descripción en un modelo intermedio y el generador produce un editor Blockly. La aportación principal no es un bloque concreto, sino el mecanismo reutilizable que permite generar editores desde descripciones de dominio de alto nivel.

Capítulo 5

Implementación de Model2Blockly

Este capítulo explica la implementación de Model2Blockly siguiendo el recorrido de una ejecución completa. A diferencia del capítulo anterior, centrado en el diseño y la arquitectura, aquí se identifican las tecnologías, clases y operaciones que materializan cada etapa de la cadena generativa.

La exposición comienza mostrando cómo ejecutar la herramienta sobre el ejemplo AppMaker y cómo localizar el editor obtenido. Después se presentan el flujo interno, la organización del código y la carga y validación de las dos entradas. Sobre esa base se detallan la ruta textual `.m2b`, el contrato y la serialización del modelo intermedio `EditorSpec`, los adaptadores Java, el generador HTML/JavaScript y el comportamiento en el navegador. El capítulo termina con la integración y distribución en Eclipse, las entradas independientes y las pruebas de implementación.

5.1 Ejecución de Model2Blockly

Antes de detallar las clases internas, esta sección presenta el procedimiento observable desde el punto de vista del usuario. La forma principal de ejecutar Model2Blockly es mediante su plugin de Eclipse. El entorno debe iniciarse con Java 21 y el plugin puede instalarse desde el repositorio p2 publicado por el proyecto o ejecutarse desde el código fuente mediante una instancia runtime de Eclipse. En ambos casos, la acción de generación se aplica a un fichero de entrada situado en el workspace.

El ejemplo AppMaker incluido en el repositorio permite reproducir la ruta Ecore me-

diante los siguientes pasos:

1. abrir o importar el proyecto principal en Eclipse;
2. seleccionar el fichero `model/app_maker.ecore`;
3. ejecutar `Model2Blockly` -> `Generate Blockly Editor` desde el menú contextual;
4. revisar el informe `app_maker_generated/generation_report.html` que Eclipse abre al finalizar la generación;
5. abrir `app_maker_generated/html/Appmaker_standalone.html` en un navegador para utilizar el editor generado.

La ruta textual se ejecuta de la misma forma seleccionando `examples/app_maker.m2b`. En los dos casos, si no se detectan errores, la herramienta crea junto a la entrada un directorio con el sufijo `_generated`. El directorio contiene el informe de generación, una serialización XMI de `EditorSpec`, las definiciones de bloques, la paleta, los scripts de generación y validación, un modelo de ejemplo y las páginas HTML del editor.

La Figura 5.1 muestra el informe que se abre después de ejecutar la acción sobre `app_maker.ecore`. En su parte superior se identifican la entrada, el paquete `Ecore`, el número de bloques y categorías, y las validaciones generadas. La tabla central permite comprobar que la ejecución ha recorrido el modelo fuente, `EditorSpec`, su serialización XMI y la generación del editor `Blockly`.

Conviene distinguir las dos ejecuciones implicadas. La primera se realiza en Eclipse y transforma la descripción del dominio en artefactos web. La segunda se realiza en el navegador al abrir la página autónoma `*_standalone.html`; en ella, el usuario construye modelos del dominio mediante bloques, ejecuta las validaciones generadas y exporta el resultado como JSON, XMI o código textual. La Figura 5.2 muestra esta segunda ejecución con el modelo de ejemplo cargado en el workspace.

Las secciones siguientes abren esta cadena paso a paso y relacionan cada operación visible con su implementación.

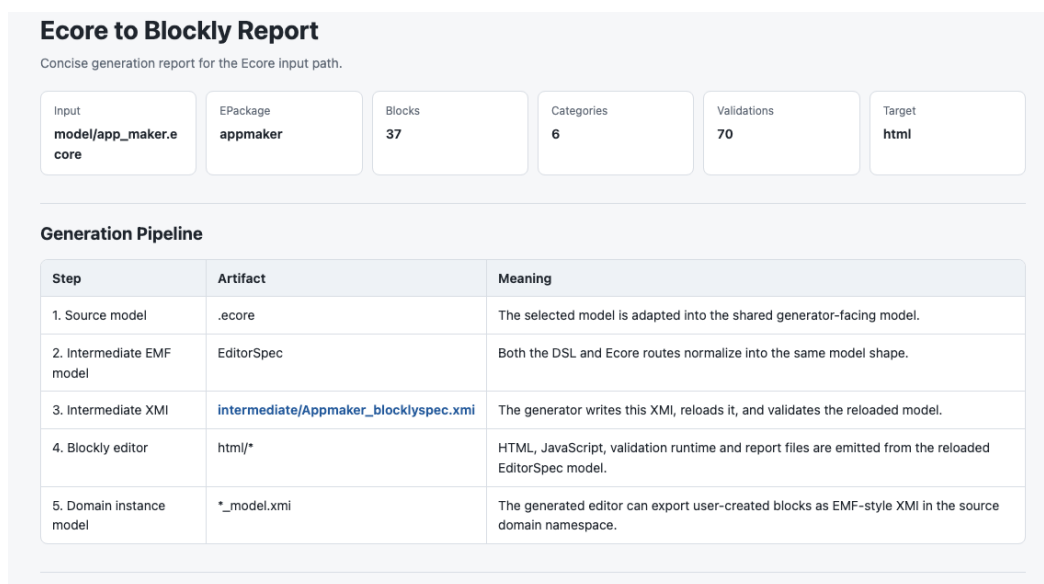


Figura 5.1: Informe abierto después de generar el editor desde `app_maker.ecore`

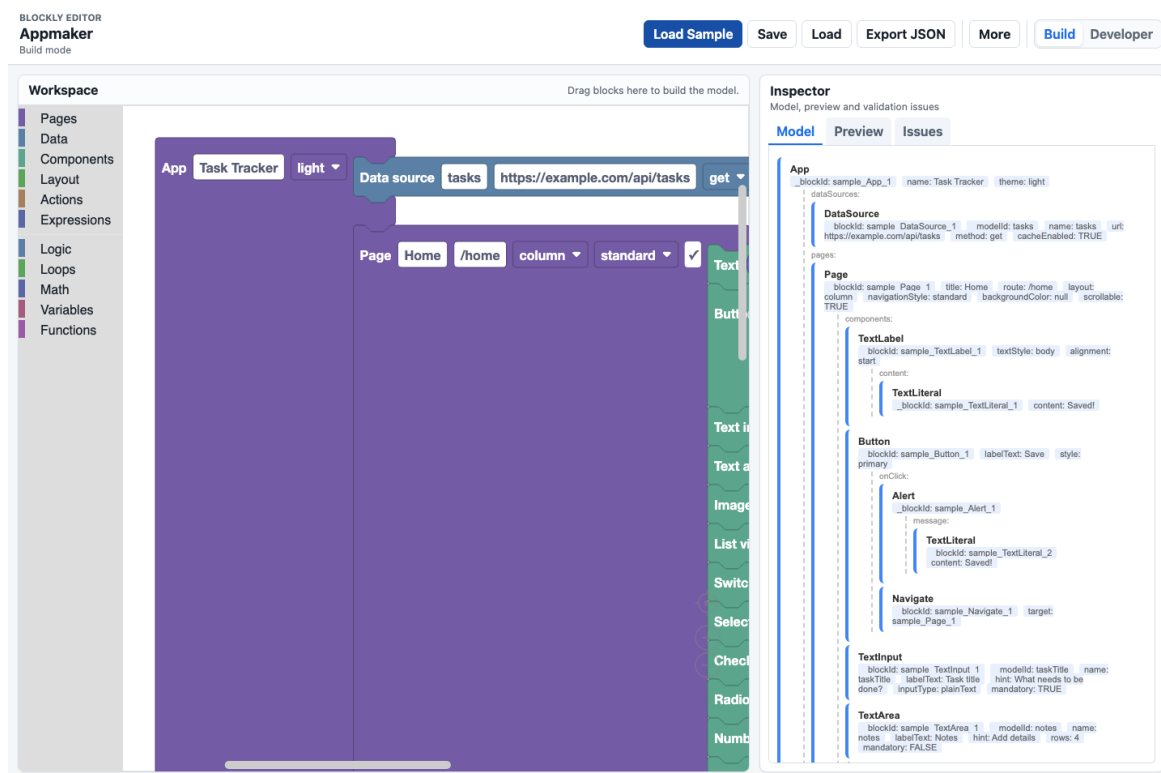


Figura 5.2: Editor Blockly ejecutado en el navegador después de la generación desde `app_maker.ecore`

5.2 Flujo interno de una ejecución

En la ejecución integrada, la acción seleccionada por el usuario se atiende en `GenerateBlocklyEditorHandler`. Esta clase determina la extensión del fichero y elige una de las dos ramas de carga. La Figura 5.3 representa esta secuencia con los nombres de los componentes implementados. No es una segunda arquitectura del sistema, sino la traza concreta que sigue una petición de generación dentro del plugin.

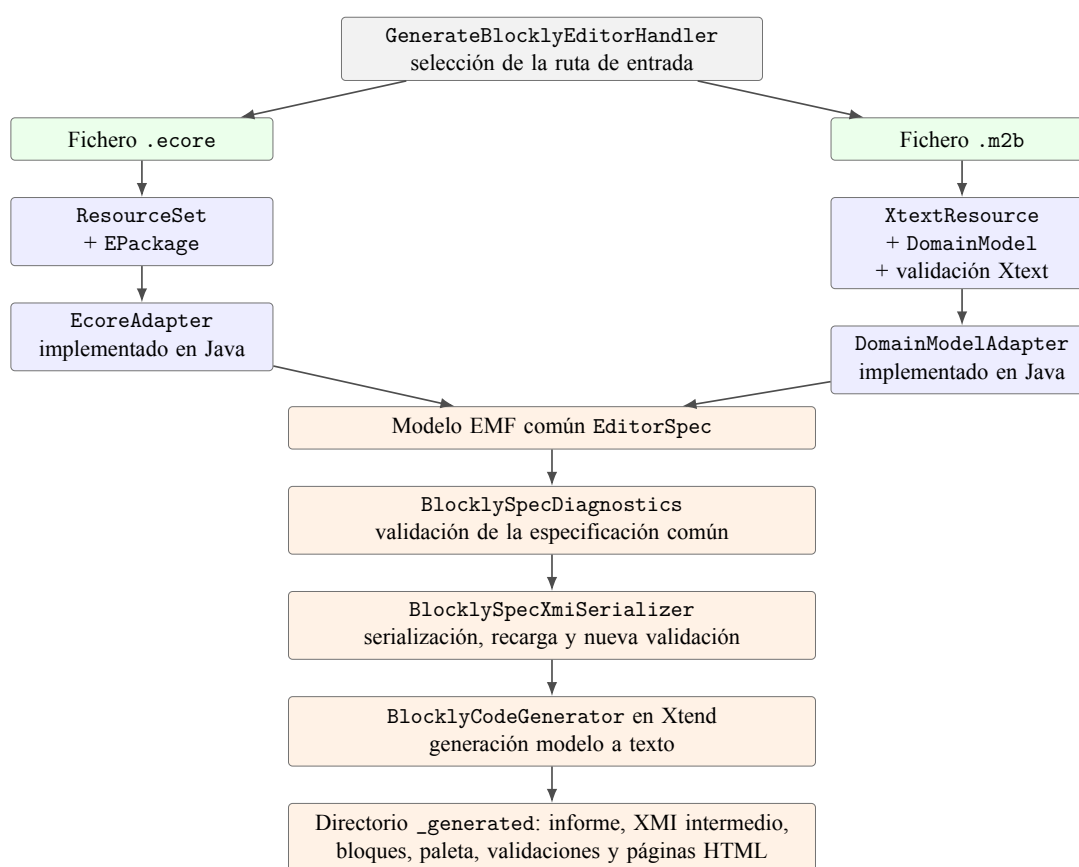


Figura 5.3: Flujo interno implementado para una petición de generación

En la rama Ecore, EMF carga el recurso y obtiene un `EPackage`; en la rama textual, Xtext analiza, enlaza y valida el fichero antes de obtener un `DomainModel`. Los adaptadores Java convierten ambos objetos en una instancia de `EditorSpec`. A partir de ese punto, la ejecución es común: se comprueba el modelo, se serializa como XMI, se vuelve a cargar y se valida de nuevo antes de entregarlo al generador Xtend.

El generador devuelve un mapa de rutas y contenidos textuales. El manejador crea el directorio de salida, escribe cada entrada del mapa y actualiza el workspace de Eclipse. Esta última escritura explica por qué una única acción produce varios ficheros relacionados y no solo la página autónoma mostrada en la sección anterior. Las secciones posteriores detallan las operaciones resumidas en cada caja de la figura.

5.3 Organización del proyecto

La implementación se distribuye en tres bundles OSGi ejecutables creados según la estructura habitual de Xtext [10]. El proyecto `io.github.plortinus.model2blockly` contiene la gramática, el modelo EMF común, los adaptadores, la validación, los generadores y las entradas de línea de comandos. El proyecto terminado en `.ide` aporta la configuración de los servicios de lenguaje independientes de la interfaz. Finalmente, el proyecto terminado en `.ui` registra el editor, los menús y los handlers que conectan la generación con el workspace de Eclipse.

Otros dos proyectos no añaden lógica a la transformación. El proyecto `.feature` agrupa los bundles instalables y `.updatesite` construye el repositorio p2 utilizado para distribuir el plugin. Dentro de los tres bundles ejecutables, la Tabla 5.1 relaciona cada componente con los ficheros que materializan su implementación y con la tecnología empleada.

Componente	Implementación concreta	Función en la ejecución
DSL textual	<code>Model2Blockly.xtext</code> (Xtext) y <code>Model2BlocklyValidator.java</code> (Java)	Analizar y validar una entrada .m2b.
Modelo EMF común	<code>blockly_editor_spec.ecore</code> y código Java generado en <code>emf-gen</code>	Representar la especificación normalizada <code>EditorSpec</code> .
Adaptadores	<code>EcoreAdapter.toEditorSpec</code> (EPackage) y <code>DomainModelAdapter.toEditorSpec</code> (DomainModel) (Java)	Recorrer cada modelo de entrada y construir el mismo <code>EditorSpec</code> .
Diagnóstico y XMI	<code>BlocklySpecDiagnostics.java</code> y <code>BlocklySpecXmiSerializer.java</code> (Java)	Validar, serializar y volver a cargar el modelo común.
Generación	<code>BlocklyCodeGenerator.xtend</code> (Xtend) y clases auxiliares del paquete <code>generator</code> (Java)	Producir HTML, JavaScript, JSON, informes y modelos de ejemplo.
Integración Eclipse	<code>GenerateBlocklyEditorHandler.java</code> (Java) y <code>plugin.xml</code>	Registrar el comando, ejecutar la cadena y escribir sus resultados en el workspace.
Ejecución standalone	Clases <code>*Main.java</code> del paquete <code>standalone</code> (Java)	Exponer la generación y el parcheado mediante línea de comandos.
Pruebas	Clases <code>*Test.java</code> del directorio <code>test</code> (JUnit/Java)	Comprobar adaptadores, modelo común, validación y generación.

Tabla 5.1: Organización de los componentes implementados

En particular, la fila *Adaptadores* corresponde a dos clases escritas manualmente en Java, no a código de transformación generado por Xtext. Sus métodos públicos reciben objetos EMF que ya han sido cargados: un `EPackage` en la primera ruta y un `DomainModel` en la segunda. Ambas clases recorren esos objetos mediante sus API tipadas y delegan la creación final de la instancia EMF de `EditorSpec` en `BlocklySpecModelMapper`. Las secciones 5.7 y 5.8 detallan las reglas aplicadas por cada clase.

Esta organización materializa la arquitectura del capítulo anterior sin repetirla: allí se justificaron los componentes y sus dependencias; aquí se identifican las clases, los lenguajes y los artefactos que ejecutan esas responsabilidades.

5.4 Carga y validación de las entradas

La carga se implementa antes de los adaptadores. `EcoreAdapter` recibe un objeto EMF ya construido y `DomainModelAdapter` recibe el modelo tipado producido por Xtext; ninguna de las dos clases abre ficheros. En la ejecución desde Eclipse, esta responsabilidad reside en `GenerateBlocklyEditorHandler`; las clases de entrada independiente reproducen el mismo patrón para ficheros situados fuera del workspace. El resultado de esta etapa es un `EPackage` o un `DomainModel` válido para iniciar la transformación.

5.4.1 Carga de un metamodelo .ecore

El método `generateFromEcore` registra `EcoreResourceFactoryImpl` para la extensión `ecore`, crea un `ResourceSetImpl` y construye una URI de fichero a partir de la ubicación del recurso seleccionado. La llamada `getResource(uri, true)` solicita la carga inmediata del documento XMI y EMF resuelve sus elementos y referencias. El primer objeto raíz se obtiene como `EPackage` y se entrega a `EcoreAdapter`. La clase `EcoreToBlocklyMain` utiliza la misma secuencia en su método `loadEPackage`.

Esta rama no utiliza el parser ni el validador de Xtext. Los problemas que impiden leer el recurso se propagan como excepciones de carga; las restricciones específicas que necesita el generador se comprueban después de la adaptación mediante `BlocklySpecDiagnostics`. Esta separación evita interpretar como sintaxis textual un metamodelo que ya dispone del formato estándar de EMF.

5.4.2 Carga y validación de una entrada .m2b

La ruta textual comienza registrando el lenguaje con `Model2BlocklyStandaloneSetup`. La configuración generada por Xtext asocia el mismo lenguaje con las extensiones `.m2b` y `.model2blockly`; la segunda se conserva por compatibilidad. El inyector de Guice proporciona el `ResourceSet`, que carga el fichero como un `XtextResource`. El parser

crea el árbol EMF y el enlazador resuelve los nombres usados para categorías, superclases y tipos. Si la carga termina correctamente, el primer elemento del recurso debe ser una instancia de `DomainModel`.

El Listado 5.1 resume las llamadas Java que implementan esta etapa en el manejador. La clase de entrada independiente `Model2BlocklyToBlocklyMain` contiene la misma secuencia, cambiando únicamente el origen de la ruta del fichero.

```
Injector injector = new Model2BlocklyStandaloneSetup()
    .createInjectorAndDoEMFRegistration();
ResourceSet resourceSet = injector.getInstance(ResourceSet.class);
Resource resource = resourceSet.getResource(fileUri, true);
XtextResource xtext = (XtextResource) resource;

xtext.validateConcreteSyntax();
Model2BlocklyValidationDiagnostics.throwIfSyntaxErrors(
    sourceLabel, resource.getErrors());
Model2BlocklyValidationDiagnostics.throwIfErrors(
    sourceLabel,
    xtext.getResourceServiceProvider().getResourceValidator()
        .validate(resource, CheckMode.ALL,
            CancelIndicator.NullImpl));

DomainModel domain =
    (DomainModel) resource.getAllContents().next();
```

Listado 5.1: Secuencia esencial de carga y validación de una entrada .m2b

La primera comprobación recoge los errores producidos durante el análisis y el enlazado. A continuación, el `IResourceValidator` ejecuta en modo `ALL` las reglas `@Check` de `Model2BlocklyValidator`. Estas reglas detectan, entre otros casos, nombres de clase duplicados, cardinalidades incoherentes, entradas de valor con un tipo no válido, widgets incompatibles, referencias a campos inexistentes y expresiones de validación no soportadas. El método `throwIfErrors` detiene la generación únicamente ante diagnósticos de severidad *error*; las advertencias permanecen visibles en el editor, pero permiten continuar.

5.4.3 Niveles de validación y comunicación de errores

La implementación aplica tres niveles consecutivos. Primero se comprueba que el recurso pueda cargarse y, en la ruta textual, que su sintaxis y sus referencias sean correctas. Después se ejecutan las reglas semánticas específicas del DSL. Finalmente, tras llamar al adaptador correspondiente, `BlocklySpecDiagnostics.assertValid` comprueba las in-

variantes del EditorSpec común. Esta última validación se repite después de serializar y recargar el XMI, por lo que el generador solo recibe un modelo que ha superado el contrato intermedio completo.

Cuando la ejecución se realiza en Eclipse, los errores semánticos y los diagnósticos del modelo común se convierten en marcadores IMarker .PROBLEM asociados al fichero y, cuando existe información de origen, a su línea. El manejador vuelve a abrir la entrada y muestra un diálogo con el resumen. En las entradas independientes, el mismo diagnóstico se escribe en la salida de error y el proceso termina con un código distinto de cero. De este modo, las dos formas de ejecución comparten las comprobaciones, aunque presentan el resultado mediante interfaces diferentes.

5.5 Ruta textual .m2b con Xtext

La gramática Xtext define el lenguaje textual de entrada .m2b y vincula sus reglas con los conceptos del metamodelo EMF asociado. A partir de esta gramática, Xtext genera el parser y el soporte de edición [10, 11, 21]. Así, cada fichero DSL puede tratarse como un modelo EMF. La raíz de ese modelo agrupa el nombre del dominio, las opciones de generación, la configuración del workspace, las categorías, las clases y las restricciones.

Las clases del dominio se declaran con una construcción específica del DSL. Cada clase puede ser abstracta, generar un bloque de salida, heredar de otra clase, pertenecer a una categoría, definir color, etiqueta, tooltip, disposición en línea y plantilla de código. Dentro de una clase se declaran cuatro tipos de elementos: atributos, contenciones, referencias y entradas de valor. Esta distinción en la gramática se corresponde con las formas principales en las que Blockly representa información: campos, entradas de sentencias, desplegables de referencia y conectores de valor.

El DSL también incluye opciones de interfaz. Con estas opciones se pueden indicar widgets, etiquetas, textos de ayuda, grupos, orden de presentación, campos de solo lectura, campos ocultos y campos usados para mostrar referencias. Esta información no es necesaria para un editor Blockly básico, pero permite enriquecer la salida generada sin escribir JavaScript a mano. En la memoria se usa para explicar la correspondencia con Ecore y para mostrar una forma más compacta de definir el mismo lenguaje AppMaker.

El Listado 5.2 muestra un fragmento representativo de la ruta textual .m2b aplicado al dominio AppMaker. En pocas líneas se declaran una categoría, un bloque raíz, una con-

tención con cardinalidad y un bloque que referencia dinámicamente una fuente de datos. La misma información acabaría convertida en campos, entradas y validaciones dentro del modelo intermedio.

```
domain Appmaker codeLanguage "javascript" codeFileExtension ".js"

category Pages label "Pages" colour 260
category Components label "Components" colour 160
category Data label "Data" colour 210

class App category Pages colour 260 label "App" {
  attribute name : string default "Task Tracker" required
  widget text uiLabel "App name" group "App" order 1
  contains DataSource dataSources [1..20]
  uiLabel "Data sources" group "Data" order 3
  contains Page pages [1..20]
  uiLabel "Pages" group "Pages" order 4
}

class ListView extends Component category Components colour 160 label "List view" {
  reference DataSource source required
  widget reference-select uiLabel "Source" referenceLabelField name
  value TextExpression itemTitle shadow TextLiteral
}
```

Listado 5.2: Fragmento representativo del DSL textual .m2b para AppMaker

Una vez analizado el fichero como se explicó en la Sección 5.4, el resultado de esta gramática es un `DomainModel` enlazado y validado. Ese objeto constituye la entrada del adaptador Java descrito posteriormente en la Sección 5.8.

5.6 Modelo intermedio EMF y vista Java

El contrato del modelo intermedio está definido como metamodelo EMF en el fichero `blockly_editor_spec.ecore`, dentro del directorio `model`. EMF genera a partir de él las interfaces Java `EditorSpec`, `BlockTypeSpec`, `FieldSpec` y los demás tipos del paquete `intermediate.blocklyspec`, junto con `BlocklySpecFactory`. Para facilitar la construcción de esas instancias, el proyecto conserva además `BlocklyEditorSpec`, una vista Java auxiliar del mismo contenido. `BlocklySpecModelMapper` implementa la copia en ambos sentidos entre esa vista y las instancias creadas por la factoría EMF.

El objeto EMF no se entrega directamente al generador después de la adaptación. La se-

rialización se encapsula en `BlocklySpecXmiSerializer`. Esta clase crea un `ResourceSet Impl` y registra `BlocklySpecPackage.eNS_URI` y añade el `EditorSpec` a un `XMIResource Impl` con la URI en memoria `memory:/blocklyspec.xmi`. El recurso se guarda en UTF-8, se vuelve a cargar desde esa cadena y se comprueba que su raíz siga siendo un `EditorSpec`. El Listado 5.3 muestra cómo el manejador aplica este ciclo en las dos rutas de entrada.

```
BlocklySpecDiagnostics.assertValid(intermediateModel, sourceLabel,
    sourceMapper);

String intermediateXmi =
    BlocklySpecXmiSerializer.toXmi(intermediateModel);
EditorSpec spec =
    BlocklySpecXmiSerializer.fromXmiToEditorSpec(intermediateXmi);

BlocklySpecDiagnostics.assertValid(
    spec, sourceLabel + "_intermediate_XMI");
Map<String, String> generated =
    new BlocklyCodeGenerator().generate(spec);
```

Listado 5.3: Validación, serialización y recarga del modelo intermedio

En el extracto, `sourceMapper` representa el mapeador creado por `forEPackage` o `forDomainModel`, según la ruta. El primer diagnóstico puede asociarse con elementos del modelo fuente mediante `sourceMapper`; el segundo valida el objeto recargado. De este modo, un error de serialización, una raíz incorrecta o la pérdida de un dato obligatorio se detectan antes de generar texto. La misma cadena XMI se conserva después como `intermediate/*_blocklyspec.xmi`, por lo que el artefacto inspeccionable es exactamente el contenido usado para generar el editor.

Cada tipo de bloque guarda su forma de conexión: bloque raíz, bloque apilable, bloque apilable tipado, bloque de valor o bloque de valor tipado. Esta clasificación permite generar las conexiones visuales adecuadas sin mezclar reglas específicas de Blockly con la lectura del dominio. La implementación mantiene además listas separadas para campos, entradas de sentencias, referencias y entradas de valor, lo que facilita generar cada tipo de elemento con la construcción Blockly adecuada.

El modelo intermedio también conserva información de orden para las entradas de un bloque. Esta decisión es relevante para la ruta `Ecore`: cuando las características de una clase se leen desde el metamodelo, el orden original de atributos y referencias puede influir en la legibilidad del bloque generado. El generador usa esta lista cuando está disponible y, si no existe, aplica un orden por defecto.

La ventaja de esta implementación es el desacoplamiento. Los adaptadores de entrada producen una instancia conforme al mismo metamodelo intermedio, y los generadores consumen su vista Java sin depender de cómo fue creada. Así, las entradas y las salidas quedan separadas en el código, pero el contrato MDE sigue siendo el metamodelo EMF y su serialización XMI.

5.7 Adaptador desde Ecore

Los dos adaptadores se implementan como clases Java ordinarias con métodos estáticos; no se instancian ni contienen plantillas de generación. El Listado 5.4 muestra sus puntos de entrada reales. Cada método realiza primero una normalización hacia `BlocklyEditorSpec`, una vista Java interna formada por listas y objetos tipados, y después invoca `BlocklySpecModelMapper.toEmfSpec`. Este segundo paso usa `BlocklySpecFactory.eINSTANCE` para crear el `EditorSpec` definido por el metamodelo común y copiar recursivamente categorías, bloques, campos, entradas, validaciones y opciones de workspace. Por tanto, la vista Java es un objeto auxiliar de construcción; el resultado contractual del adaptador es la instancia EMF `EditorSpec`.

```
// EcoreAdapter.java
public static EditorSpec toEditorSpec(EPackage pkg) {
    return BlocklySpecModelMapper.toEmfSpec(fromEPackage(pkg));
}

// DomainModelAdapter.java
public static EditorSpec toEditorSpec(DomainModel domain) {
    return BlocklySpecModelMapper.toEmfSpec(fromDomainModel(domain));
}
```

Listado 5.4: Puntos de entrada Java de los dos adaptadores

En la ruta Ecore, `fromEPackage` recibe el objeto obtenido durante la carga descrita en la Sección 5.4. El método copia primero el nombre, `nsURI`, prefijo y opciones globales del paquete. Después reúne los nombres de las clases que son destino de alguna referencia de contención; este conjunto se usa más adelante para distinguir bloques raíz de bloques que deben conectarse como hijos. A continuación recorre todos los `EClassifier` del paquete y de sus subpaquetes, selecciona los `EClass` y llama a `convertEClass`. Este proceso usa directamente la API de EMF: clases, atributos, referencias, herencia y cardinalidades [8].

Desde el punto de vista de MDE, este adaptador es una transformación de modelos implementada en Java sobre la API de EMF. No se ha usado un lenguaje específico de transformación como ATL o ETL. La razón es pragmática: el generador debe integrarse dentro de un plugin Eclipse/Xtext, reutilizar utilidades Java del proyecto, emitir diagnósticos concretos sobre el fichero de entrada y construir la instancia EMF `EditorSpec` que después se serializa como XMI. La implementación queda así dentro de la misma cadena Java que carga el recurso y comunica los errores al entorno Eclipse. La Tabla 5.2 resume las reglas principales de la transformación.

El mapeo estructural se implementa de forma automática. Un atributo `Ecore` se transforma en un campo visual; su tipo determina si el campo Blockly será textual, numérico, booleano, desplegable, color, ángulo o imagen. Una referencia de contención se transforma en una entrada estructural, porque representa una relación jerárquica entre elementos del modelo. Una referencia no contenida se transforma en una referencia dinámica, porque apunta a otro elemento ya existente. Las cardinalidades se convierten en reglas de validación para que el editor generado pueda avisar al usuario si faltan elementos obligatorios, si se supera un límite superior o si un campo requerido queda sin completar.

La inferencia de conexiones también se realiza en el adaptador. Las clases contenedoras que no están contenidas por otras pueden convertirse en bloques raíz sin conexión vertical. Las clases que heredan de una superclase pueden generar conexiones tipadas. Las clases marcadas como bloques de salida generan conectores de valor. Esta inferencia es importante porque evita que el usuario tenga que escribir a mano reglas de conexión Blockly para cada clase.

Elemento Ecore de entrada	Elemento EditorSpec destino	Uso en Blockly
EPackage	EditorSpec con nombre, nsURI y opciones globales	Identidad del editor y exportación XMI.
EClass concreta	BlockTypeSpec	Tipo de bloque disponible en la paleta.
EClass abstracta	BlockTypeSpec abstracto	Tipo usado para conexiones, pero no instanciable.
EAttribute	FieldSpec	Campo de texto, número, booleano, color, ángulo o desplegable.
EReference containment=true	StatementInputSpec	Entrada vertical para bloques hijos.
EReference containment=false	ReferenceFieldSpec	Desplegable dinámico hacia elementos existentes.
lowerBound/upperBound	ValidationSpec y límites de entrada	Avisos de obligatoriedad y cardinalidad.
EAnnotation	Metadatos de bloque, interfaz, validación o código	Color, categoría, etiqueta, tooltip, widget y plantilla.

Tabla 5.2: Transformación principal de Ecore a EditorSpec

El adaptador permite enriquecer la generación mediante anotaciones Ecore. Las anotaciones con fuente `blockly` afectan a la sintaxis visual: etiqueta, color, categoría, tooltip, disposición en línea, bloque de salida, rangos numéricos, entradas de valor o `shadow blocks`. Las anotaciones con fuente `ui` se incorporan al modelo intermedio para personalizar la salida HTML. Las anotaciones con fuente `code` permiten definir lenguaje, extensión de archivo y plantillas de generación. Así, un metamodelo sin anotaciones puede generar un editor básico, mientras que un metamodelo anotado puede producir una experiencia más adaptada al dominio.

El Listado 5.5 muestra el mismo tipo de información en la ruta Ecore. El metamodelo conserva las clases y referencias EMF, y las anotaciones añaden la sintaxis visual, la regla de validación y la plantilla de código usadas por el generador.

```
<eClassifiers xsi:type="ecore:EClass" name="Navigate" eSuperTypes="#//Action">
  <eAnnotations source="blockly">
    <details key="colour" value="30"/>
    <details key="category" value="Actions"/>
    <details key="inputsInline" value="true"/>
  </eAnnotations>
  <eAnnotations source="validation">
    <details key="mustFollow" value="Alert"/>
  </eAnnotations>
  <eAnnotations source="code">
    <details key="template" value="navigate(&quot;{{target}}&quot;);"/>
  </eAnnotations>
  <eStructuralFeatures xsi:type="ecore:EReference" name="target"
    lowerBound="1" eType="#//Page">
    <eAnnotations source="ui">
      <details key="widget" value="reference-select"/>
      <details key="label" value="Target page"/>
      <details key="referenceLabelField" value="title"/>
    </eAnnotations>
  </eStructuralFeatures>
</eClassifiers>
```

Listado 5.5: Fragmento de `model/app_maker.ecore` con anotaciones

5.8 Adaptador desde la ruta textual .m2b

El adaptador de la ruta textual también está implementado manualmente en Java. `DomainModelAdapter.fromDomainModel` recibe la raíz EMF producida por Xtext y ejecuta, en orden, los métodos `convertCategories`, `convertClasses`, `convertConstraints`, `convertValidations` y `convertWorkspaceOptions`. Después genera las validaciones inferidas de cardinalidad, obligatoriedad y unicidad. El objeto Java resultante vuelve al método `toEditorSpec` del Listado 5.4, donde se convierte al mismo modelo EMF común que en la ruta Ecore.

El mapeo es más directo que en Ecore porque la gramática ya distingue los conceptos específicos que necesita Model2Blockly. Los atributos se convierten en campos, las contenciones en entradas estructurales, las referencias en despleables dinámicos y las entradas de valor en conexiones horizontales. Las restricciones de orden se convierten en validaciones, y los campos marcados como obligatorios generan comprobaciones de obligatoriedad.

La conversión de cada clase se implementa con un recorrido de la lista `Feature` generada

por Xtext. El Listado 5.6 muestra el despacho esencial. Cada tipo concreto se entrega a un método especializado y el nombre de la entrada se añade a `orderedInputNames`; de este modo, el orden escrito en el fichero `.m2b` se conserva al construir el bloque.

```
for (Feature feature : cls.getFeatures()) {
    if (feature instanceof Attribute) {
        FieldSpec field = convertAttribute((Attribute) feature);
        bt.getFields().add(field);
        bt.getOrderedInputNames().add(field.getName());
    } else if (feature instanceof Containment) {
        StatementInputSpec input =
            convertContainment((Containment) feature);
        bt.getStatementInputs().add(input);
        bt.getOrderedInputNames().add(input.getName());
    } else if (feature instanceof Reference) {
        ReferenceFieldSpec reference =
            convertReference((Reference) feature);
        bt.getReferenceFields().add(reference);
        bt.getOrderedInputNames().add(reference.getName());
    } else if (feature instanceof ValueInput) {
        ValueInputSpec input = convertValueInput((ValueInput) feature);
        bt.getValueInputs().add(input);
        bt.getOrderedInputNames().add(input.getName());
    }
}
```

Listado 5.6: Despacho de elementos del DSL en DomainModelAdapter

Elemento del DSL	Elemento EditorSpec destino	Uso en Blockly
domain	EditorSpec	Nombre del editor y opciones de generación.
category	CategorySpec	Categoría de la paleta y color por defecto.
class	BlockTypeSpec	Tipo de bloque.
attribute	FieldSpec	Campo visual del bloque.
contains [m..n]	StatementInputSpec y ValidationSpec	Entrada de hijos y comprobación de cardinalidad.
reference	ReferenceFieldSpec	Desplegable dinámico.
value	ValueInputSpec	Entrada horizontal para bloques de valor.
must follow	ValidationSpec	Regla local de orden.

Tabla 5.3: Normalización del DSL textual `.m2b` a EditorSpec

La implementación usa también información de herencia para determinar la forma de conexión. Una clase marcada como bloque de salida se convierte en bloque de valor; una clase con superclase genera conexiones tipadas; una clase con contenciones y sin superclase puede funcionar como bloque raíz; y una clase ordinaria se genera como bloque apilable. De este modo, la semántica declarada en la sintaxis textual se transforma en reglas visuales de Blockly sin que el autor del dominio tenga que conocer la API de bajo nivel de la biblioteca.

La estructura principal se lee con los tipos Java generados por Xtext (ClassDef, Attribute, Containment, Reference y ValueInput). Solo algunas opciones de interfaz, código y workspace se obtienen dinámicamente con `eClass().getEStructuralFeature` y `eGet`. Esta decisión permite que metadatos añadidos recientemente a la gramática puedan leerse antes de regenerar todos los accesores, sin renunciar a la comprobación estática en el recorrido estructural que determina los bloques.

5.9 Generador HTML/JavaScript

El generador de la salida HTML/JavaScript transforma el modelo intermedio en los artefactos necesarios para abrir el editor en un navegador: bloques, paleta, exportación, validaciones, página de edición, página autónoma, workspace de validaciones, modelo de ejemplo e informe de generación. Esta salida usa Blockly para definir bloques, inyectar el workspace, gestionar la paleta, serializar y generar código [4].

La generación de código se implementa principalmente en Xtend. La clase principal, `BlocklyCodeGenerator.xtend`, recibe un `EditorSpec` y produce un mapa ordenado de nombres de fichero a contenido textual. El Listado 5.7 reproduce el método de entrada: cada función `generate*` devuelve una cadena y el mapa conserva la relación entre esa cadena y su fichero de destino.

Listado 5.7: Método de entrada de `BlocklyCodeGenerator.xtend`

```
def Map<String, String> generate(EditorSpec spec) {
    val files = new LinkedHashMap<String, String>()
    val base = spec.domainName ?: "domain"
    files.put(base + "_blocks.js", generateBlocksJs(spec))
    files.put(base + "_toolbox.js", generateToolboxJs(spec))
    files.put(base + "_generators.js", generateGeneratorsJs(spec))
    files.put(base + "_validations.js", generateValidationsJs(spec))
    files.put(base + "_editor.html", generateEditorHtml(spec))
    files.put(base + "_standalone.html", generateStandaloneHtml(spec))
}
```

```

files.put("validation_workspace.html",
    ValidationWorkspaceHtmlGenerator.generate(spec))
files.put("validation_blocks.json",
    ValidationBlockModelGenerator.generate(spec))
files.put("validation_runtime.js",
    ValidationRuntimeGenerator.generateClassicJs(spec))
files.put("sample_model.json", SampleModelGenerator.generate(spec))
return files
}

```

Xtend se usa como mecanismo de generación porque se integra bien con Xtext y permite escribir plantillas de texto legibles dentro del proyecto Eclipse. No se ha usado Acceleo ni un motor externo de plantillas. Algunas piezas auxiliares, como el modelo de ejemplo, el runtime de validaciones y la vista visual de reglas, se implementan en clases Java separadas para mantener responsabilidades acotadas.

El mapa no escribe directamente en disco. El `GenerateBlocklyEditorHandler` antepone `html/` a sus rutas y añade tres resultados producidos fuera de Xtend: el XMI intermedio, el `README.md` y el informe de generación. Finalmente, `writeFiles` crea los `IFile` del workspace y actualiza el proyecto. Esta separación permite reutilizar el mismo generador desde las entradas standalone sin depender de la API de Eclipse.

La generación de bloques comienza en `generateBlocksArray`, que filtra los tipos concretos y llama a `blockTypeToBlockJson`. Este método crea mapas de campos, referencias y entradas, y recorre `orderedInputNames` para conservar el orden establecido por el adaptador. Cada elemento se delega en `fieldToArg`, `referenceToArg`, `valueInputToArg` o `statementInputToArg`. Finalmente, un switch sobre `ConnectionType` emite `previousStatement`, `nextStatement` u `output` con tipo libre o tipado. Las clases abstractas quedan en el modelo para resolver herencia y conexiones, pero no se incluyen como bloques instanciables.

La generación de la paleta produce el conjunto de bloques disponible para el usuario. Si no hay categorías, el generador puede emitir una paleta simple. Si existen categorías, genera una paleta categorizada, incluyendo subcategorías cuando se han definido. También se incluyen categorías integradas de Blockly, como lógica, bucles, matemáticas, variables y funciones, para que los dominios puedan combinar bloques propios con construcciones generales cuando sea necesario.

Se generan dos envoltorios HTML sobre esos mismos contenidos. La página `*_editor.html` carga por separado los ficheros de bloques, paleta, generadores y validaciones, lo que

facilita incrustarla y depurar cada módulo. La página `*_standalone.html` incrusta el JavaScript específico del dominio en un único documento para simplificar la demostración. En ambos casos, la biblioteca Blockly y su generador JavaScript se cargan desde la CDN indicada en las etiquetas `script`; por ello, la página autónoma no contiene una copia local de Blockly y requiere acceso a esa dependencia al abrirse.

La Tabla 5.4 resume los ficheros principales que emite la cadena generativa. Esta lista permite interpretar la salida como un resultado reproducible y no como una única página HTML aislada.

Artefacto generado	Contenido
<code>html/*_blocks.js</code>	Definiciones de bloques Blockly.
<code>html/*_toolbox.js</code>	Categorías y paleta del editor.
<code>html/*_generators.js</code>	Funciones de serialización y generación de código por bloque.
<code>html/*_validations.js</code>	Reglas de validación ejecutadas sobre el workspace.
<code>html/*_editor.html</code>	Página embebible del editor generado.
<code>html/*_standalone.html</code>	Página única que incrusta el código del dominio y carga Blockly desde CDN.
<code>html/sample_model.json</code>	Modelo de ejemplo usado por el botón Load Sample.
<code>html/validation_workspace.html</code>	Editor visual de reglas de validación.
<code>html/validation_blocks.json</code>	Árbol serializado de bloques de validación.
<code>html/validation_runtime.js</code>	Compilación y evaluación de las reglas visuales en el navegador.
<code>intermediate/*_blocklyspec.xmi</code>	Instancia XMI de EditorSpec recargada antes de generar HTML.
<code>generation_report.html</code>	Informe de trazabilidad de la generación.
<code>README.md</code>	Instrucciones para abrir y revisar los resultados.

Tabla 5.4: Artefactos principales emitidos por la cadena generativa

Entre esos artefactos, la exportación tiene dos objetivos. Por un lado, serializa el workspace como una estructura JSON con el tipo de bloque, identificador, campos, referencias, valores y sentencias. También puede exportar ese contenido como XMI de instancia del dominio, usando el prefijo y el URI del paquete de entrada y evitando mezclar ese XMI con el XMI intermedio EditorSpec. Por otro lado, incorpora una ruta de generación de código basada en plantillas. Si un bloque define una plantilla, el generador sustituye marcadores por valores de campos, entradas de valor o secuencias de sentencias. Si no existe plantilla,

se usa una representación textual genérica. Esta estrategia mantiene la exportación operativa incluso cuando el dominio todavía no ha definido una semántica de código completa.

5.10 Validaciones y referencias en tiempo de ejecución

La implementación distingue entre validación estática del DSL y validación en tiempo de ejecución del editor generado. La primera se realiza en Eclipse/Xtext antes de generar. La segunda se ejecuta en el navegador sobre el workspace Blockly.

`generateValidationsJs` produce las funciones `computeValidationWarnings`, `applyValidationWarnings` e `initValidations`. La primera recorre `workspace.getAllBlocks(false)` y evalúa las reglas aplicables a cada tipo. Las reglas de orden consultan `getPreviousBlock`; las de cardinalidad siguen la cadena iniciada en `getInputTargetBlock`; y las de obligatoriedad y unicidad leen y comparan los valores de campo. La segunda función agrupa los mensajes por bloque, llama a `setWarningText` y actualiza el panel de problemas. `initValidations` registra finalmente un listener para repetir el cálculo después de crear, borrar, mover o modificar un bloque.

Las reglas se representan también como árboles en `validation_blocks.json`. La clase Java `ValidationRuntimeGenerator` transforma cada árbol en una expresión de runtime y genera `validation_runtime.js`. En el navegador, `createValidationAccessors` ofrece operaciones acotadas como `value`, `has`, `inputCount`, `fieldUnique` y `previousBlockIs`; `evaluateRuntimeExpression` ejecuta la expresión con esos accesores. Esta representación permite que `validation_workspace.html` muestre las mismas reglas que utiliza el editor, en lugar de mantener una segunda definición visual independiente.

Las referencias no contenidas se implementan como campos cuyos valores son identificadores de bloques. La función generada `updateReferenceDropdowns` agrupa las instancias existentes por tipo, construye para cada destino una pareja etiqueta–identificador y sustituye las opciones del desplegable. La etiqueta se toma del `referenceLabelField`, del campo `name` o, como último recurso, del tipo y una parte del identificador. Si se elimina el bloque referenciado, el valor deja de estar entre las opciones válidas y se limpia.

Esta implementación mantiene la diferencia entre contención y referencia. Una contención crea estructura jerárquica en el workspace, mientras que una referencia selecciona un elemento que ya existe. La separación conserva la información relacional del modelo de dominio.

Cuando existe una referencia opuesta, `synchronizeOppositeReferences` calcula los valores inversos y actualiza el bloque destino. Una bandera `__syncingOppositeReferences` y la desactivación temporal de eventos evitan que esa escritura genere un ciclo de notificaciones. El Listado 5.8 resume el listener compartido que mantiene referencias, validaciones y vista de salida después de cada cambio estructural.

```
workspace.addChangeListener(function(event) {
    if (event.type === Blockly.Events.BLOCK_MOVE ||
        event.type === Blockly.Events.BLOCK_CREATE ||
        event.type === Blockly.Events.BLOCK_DELETE ||
        event.type === Blockly.Events.BLOCK_CHANGE) {
        updateReferenceDropdowns(workspace);
        synchronizeOppositeReferences(workspace, event);
        if (typeof applyValidationWarnings === 'function')
            applyValidationWarnings(workspace);
        if (typeof updateOutput === 'function') updateOutput();
    }
});
```

Listado 5.8: Actualización reactiva del editor generado

Antes de exportar JSON, XMI o código, `confirmExportIfInvalid` vuelve a ejecutar las validaciones y solicita confirmación si quedan advertencias. El usuario puede por tanto inspeccionar un modelo incompleto sin bloquear la edición, pero la exportación no oculta sus problemas. Los límites numéricos `min` y `max` se incorporan directamente a los campos Blockly.

La página incluye además un ejecutor genérico de inspección.

El método `generateRuntimeScript` genera su código a partir del modelo. Durante la ejecución, este código convierte el workspace a la representación JSON del dominio, recorre recursivamente sus nodos y resalta el bloque visitado con controles de ejecución, pausa y paso. No ejecuta la semántica particular de cada dominio; sirve para observar la estructura construida y comprobar el orden de recorrido del modelo.

El ciclo de validación se completa con una utilidad de parche. Desde línea de comandos puede ejecutarse sobre un fichero `validation_blocks.edited.json` y desde Eclipse está expuesta como `Apply Validation Blocks to Source`. El parche se aplica de forma conservadora: solo actualiza reglas que se pueden mapear de manera inequívoca a la sintaxis textual o a anotaciones Ecore, y deja fuera expresiones que no tengan una traducción directa.

El comportamiento generado sigue siendo deliberadamente ligero. No constituye un motor OCL completo ni un verificador semántico general. Cubre cardinalidad, obligatoriedad,

unicidad, referencias opuestas, expresiones declaradas y el subconjunto de invariantes OCL que los adaptadores pueden traducir a los accesorios disponibles en JavaScript.

5.11 Integración con Eclipse y distribución

La integración con Eclipse se declara en el fichero `plugin.xml` del bundle de interfaz. Este fichero registra los comandos `generateBlocklyEditor` y `applyValidationPatch`, los asocia con `GenerateBlocklyEditorHandler` y `ApplyValidationPatchHandler`, y limita su visibilidad a selecciones de un único fichero compatible. Las contribuciones al menú contextual incluyen `.ecore` y `.m2b`; la extensión histórica `.model2blockly` se conserva para poder abrir proyectos anteriores.

`GenerateBlocklyEditorHandler` extiende `AbstractHandler`. Su método `execute` obtiene el `IFile` seleccionado y delega el trabajo en un `IRunnableWithProgress` ejecutado por el servicio de progreso de Eclipse. Dentro de ese runnable se selecciona la ruta `Ecore` o la ruta textual. A continuación se reutilizan la carga de la Sección 5.4, los adaptadores de las Secciones 5.7 y 5.8, y el generador de la Sección 5.9. El resultado es el mapa de artefactos que recibe `writeFiles`; este método crea de forma recursiva el directorio `*_generated`, convierte cada contenido a UTF-8 y usa `IFile.create` o `IFile.setContents` según exista o no el recurso. Al terminar, el proyecto se refresca y se abre `generation_report.html` en el workbench.

La integración también traduce los fallos de la transformación al mecanismo de diagnósticos del IDE. Si falla la validación de `EditorSpec`, el `BlocklySpecSourceMapper` permite crear un `IMarker.PROBLEM` sobre el elemento de origen, incluyendo línea, ubicación, mensaje y sugerencia cuando están disponibles. Los errores producidos por Xtext se convierten del mismo modo en marcadores asociados al fichero `.m2b`. Por tanto, el usuario no recibe solamente una excepción Java: el editor de origen se abre y Eclipse muestra la ubicación del problema.

`ApplyValidationPatchHandler` implementa el camino inverso para las reglas editadas visualmente. El manejador propone los ficheros `validation_blocks.json` de la salida correspondiente, ejecuta primero `ValidationPatchService.dryRun` y presenta los cambios antes de pedir confirmación. Solo después ejecuta `apply`, refresca el recurso y vuelve a abrir la fuente. Cuando la escritura se realiza sobre el mismo fichero, el servicio conserva previamente una copia con extensión `.bak`.

La distribución instalable se materializa en dos niveles. El fichero `feature.xml` agrupa los bundles principal, IDE y UI en la feature `io.github.plortinus.model2blockly.feature`. Después, `category.xml` publica esa feature en la categoría `Model2Blockly` del repositorio p2. Este repositorio contiene los JAR de los tres bundles y los metadatos `content.jar` y `artifacts.jar`; de esta forma, la misma implementación descrita en este capítulo puede instalarse mediante `Help -> Install New Software` sin importar los proyectos fuente.

El repositorio publica también el código fuente, el repositorio p2 y dos demostraciones de AppMaker. Las fuentes son `model/app_maker.ecore` y `examples/app_maker.m2b`; las salidas versionadas de `examples/generated/` incluyen el informe, el XMI intermedio, el modelo de ejemplo, las reglas de validación y el editor. Así pueden inspeccionarse sin repetir la transformación. La Tabla 5.5 reúne los puntos de acceso utilizados para la entrega.

Artefacto	Ubicación
Código fuente	https://github.com/Plortinus/model2blockly
Repositorio p2	https://plortinus.github.io/model2blockly/update-site/
Sitio del proyecto	https://plortinus.github.io/model2blockly/
Demo AppMaker Ecore	https://plortinus.github.io/model2blockly/app_maker_ecore/Appmaker_standalone.html
Demo AppMaker .m2b	https://plortinus.github.io/model2blockly/app_maker_dsl/Appmaker_standalone.html
Documentación de entrega	README.md, RUNNING_EXAMPLE.md y DOCS.md

Tabla 5.5: Artefactos de entrega del repositorio Model2Blockly

5.12 Entradas independientes (standalone)

El bundle principal expone tres entradas Java ejecutables sin el workbench, resumidas en el Listado 5.9.

```
EcoreToBlocklyMain <file.ecore> [output-dir]
Model2BlocklyToBlocklyMain <file.m2b> [output-dir]
ValidationPatchMain --source <file> --validation <json> [options]
```

Listado 5.9: Interfaces de los puntos de entrada standalone

EcoreToBlocklyMain es el punto de entrada de la ruta Ecore. Registra la factoría de recursos, carga el EPackage y ejecuta EcoreAdapter. Después valida y recarga el XMI intermedio antes de pasar el resultado a BlocklyCodeGenerator.

Model2BlocklyToBlocklyMain implementa la ruta textual. Crea el inyector con el setup standalone generado por Xtext, obtiene un XtextResource, ejecuta la validación sintáctica y semántica y llama a DomainModelAdapter. A partir de EditorSpec, ambas clases reutilizan exactamente la misma generación final.

Si se omite el segundo argumento, las dos entradas crean junto a la fuente el directorio <name>_generated. Los artefactos Blockly se escriben bajo html/; el XMI intermedio, el README y el informe se añaden en las mismas rutas que usa el plugin. Durante la escritura se crean los directorios necesarios y se informa por consola de cada fichero producido. Los códigos de salida distinguen argumentos no válidos, objetivos no soportados, fallos de carga o validación y errores de escritura, lo que permite integrar estas clases en scripts de construcción.

ValidationPatchMain reutiliza ValidationSourcePatcher para aplicar el flujo de retorno de validaciones. El modo predeterminado es --dry-run; --apply escribe los cambios y --out permite conservar intacta la fuente original. Esta tercera entrada se utiliza también para comprobar automáticamente que un modelo parcheado puede volver a generar el editor.

5.13 Pruebas de implementación

La implementación incluye 142 métodos de prueba JUnit 5. El recuento se realiza sobre las seis clases del directorio test y se distribuye como muestra la Tabla 5.6.

Clase de prueba	Métodos @Test
BlocklyCodeGeneratorTest	60
BlocklyEditorSpecTest	27
DomainModelAdapterTest	6
EcoreAdapterTest	39
GeneratedBlocklySpecModelTest	5
Model2BlocklyValidatorBridgeTest	5
Total	142

Tabla 5.6: Distribución de las pruebas JUnit de implementación

Las pruebas del modelo intermedio verifican valores por defecto, tipos de campo, conexiones, filtrado de bloques abstractos y serialización XMI. Las pruebas de los adaptadores comprueban clases, atributos, referencias, herencia, cardinalidades, anotaciones y metadatos de código para las dos rutas. El grupo más amplio corresponde al generador: comprueba los nombres de artefacto, la estructura HTML, los bloques, la paleta, las validaciones, las referencias, la persistencia del workspace y las exportaciones JSON, XMI y código. La clase de puente de validación crea recursos Xtext mediante el setup standalone y verifica que los diagnósticos del DSL se propaguen antes de adaptar.

El fichero `package.json` declara además verificaciones ejecutables. `verify:plugin` inspecciona manifiestos, `plugin.xml`, la feature, el repositorio p2, los JAR empaquetados y los ejemplos generados. `verify:patch` comprueba el modo de simulación, la aplicación del parche, su idempotencia y una nueva generación Ecore. `verify:domain-xmi` valida con EMF el XMI de dominio exportado, mientras que `smoke` abre con Playwright las salidas AppMaker de Ecore y `.m2b`, carga el modelo de ejemplo y comprueba el estado observable del editor. De este modo se prueban por separado los contratos Java, el empaquetado Eclipse y la ejecución real en navegador.

5.14 Resumen de implementación

La implementación de Model2Blockly permite definir un dominio mediante Ecore anotado o mediante la ruta textual `.m2b`. Ambas entradas se transforman en el mismo `EditorSpec`, se validan antes y después de su serialización XMI y reutilizan el generador HTML/JavaScript. A partir de ese contrato se producen bloques, paleta, validaciones, referencias dinámicas, exportadores, informes, un workspace visual de validaciones y plantillas de código.

La misma cadena se expone mediante comandos de Eclipse y entradas Java independientes. La feature y el repositorio p2 distribuyen el plugin; las pruebas JUnit, de empaquetado y de humo en navegador cubren niveles complementarios. Así, la implementación materializa alrededor del modelo intermedio la arquitectura generativa definida en el capítulo anterior.

Capítulo 6

Ejemplo conductor y evaluación

La evaluación de Model2Blockly busca comprobar si el enfoque es viable. En concreto, se analiza si es posible generar editores Blockly reutilizables a partir de descripciones de dominio, sin programar cada editor desde cero. Para ello se usa un ejemplo conductor, verificaciones técnicas y pruebas automáticas.

No se evalúan aprendizaje ni usabilidad con usuarios finales. Ese tipo de estudio necesitaría participantes, tareas y métricas de interacción. Aquí la evaluación se centra en la ingeniería de la solución: rutas Ecore y .m2b, artefactos generados, validaciones, modelo intermedio y pruebas automáticas.

El capítulo comienza con la estrategia de evaluación y la presentación del ejemplo conductor. A continuación, se describe AppMaker mediante las dos rutas de entrada y se analiza la cobertura funcional alcanzada. Después se presentan las verificaciones técnicas, los resultados frente a los objetivos y la correspondencia entre el metamodelo y el editor Blockly generado. Finalmente, se discuten las limitaciones del estudio y se sintetizan sus conclusiones.

6.1 Estrategia de evaluación

La estrategia de evaluación combina cuatro tipos de evidencia:

1. un ejemplo conductor, AppMaker, definido como metamodelo Ecore anotado y como fichero textual .m2b;
2. inspección y verificación sintáctica de los artefactos generados;

3. pruebas automáticas sobre modelo intermedio, adaptadores y generadores;
4. artefactos de entrega reproducibles, incluyendo update site p2, demos desplegadas e informes de generación.

La Figura 6.1 muestra cómo se relacionan estas fuentes de evidencia. El dominio se expresa como `app_maker.ecore` y como `app_maker.m2b`. Ambas entradas convergen en la cadena `EditorSpec–HTML/JavaScript` y después se contrastan mediante informes, comprobaciones sintácticas y pruebas automáticas.

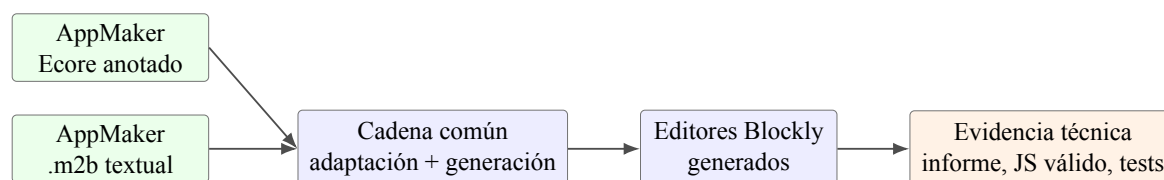


Figura 6.1: Flujo de evaluación aplicado al ejemplo conductor AppMaker

Estas fuentes responden a cuatro preguntas. Primero, si la ruta Ecore puede generar un editor Blockly funcional desde un metamodelo anotado. Segundo, si la ruta textual `.m2b` puede describir el mismo lenguaje de bloques y normalizarse hacia el mismo modelo intermedio. Tercero, si los artefactos generados son técnicamente coherentes y pueden verificarse de forma repetible. Cuarto, si el resultado puede entregarse como herramienta instalable y como demo web revisable.

La primera fuente analiza la reutilización de la cadena generativa. Un generador de editores de bloques no resulta convincente si la salida es un HTML preparado manualmente o si cada entrada exige un generador distinto. Por ello, AppMaker se describe por las dos rutas: como metamodelo Ecore con anotaciones y como fichero textual `.m2b`. Ambas formas se transforman a `EditorSpec`; la ruta Ecore y la ruta textual cuentan con salidas generadas en el repositorio y con demos desplegadas públicamente para la revisión externa del editor final.

La segunda fuente comprueba que la generación produce ficheros revisables y sintácticamente válidos. En particular, los ficheros JavaScript generados para las salidas incluidas se han comprobado con `node --check`, lo que permite detectar errores de sintaxis sin ejecutar el editor completo en el navegador. Esta comprobación no sustituye a una prueba funcional interactiva, pero sí reduce el riesgo de que el generador emita código JavaScript mal formado.

La tercera fuente son las pruebas JUnit del proyecto. Estas pruebas no dependen de la interacción manual con el navegador y permiten comprobar comportamientos internos: mapeos desde Ecore, normalización desde la ruta textual `.m2b`, comportamiento del modelo intermedio y generación HTML/JavaScript.

6.2 Ejemplo conductor seleccionado

El ejemplo seleccionado es AppMaker. Representa una aplicación visual con páginas, componentes, acciones, navegación y fuentes de datos. Se ha elegido porque combina elementos estructurales y semánticos suficientes para evaluar la generación: contenciones, herencia, clases abstractas, referencias dinámicas, campos obligatorios, entradas de valor, validaciones simples y plantillas de código.

La Tabla 6.1 resume las dos especificaciones usadas para el ejemplo conductor.

Elemento	Papel	Rasgos evaluados
AppMaker	Ecore anotado	Metamodelo EMF con clases, atributos, referencias, enumeraciones, cardinalidades y anotaciones <code>blockly</code> , <code>ui</code> , <code>code</code> y <code>runtime</code> .
AppMaker	DSL textual <code>.m2b</code>	Notación compacta para definir el mismo lenguaje de bloques mediante categorías, clases, contenciones, referencias, validaciones y metadatos sin editar directamente XML Ecore.

Tabla 6.1: Especificaciones usadas para el ejemplo AppMaker

Métrica del fichero app_maker.m2b	Valor	Interpretación
Categorías	6	Secciones principales del toolbox generado.
Clases totales	20	Conceptos declarados en la definición textual.
Clases abstractas	3	Familias de tipos usadas para conexiones.
Clases de salida	4	Bloques de expresión conectables a entradas de valor.
Atributos	30	Campos editables o exportables.
Referencias	6	Selectores dinámicos hacia elementos existentes.
Contenciones	8	Entradas estructurales para bloques hijos.
Entradas de valor	6	Conectores horizontales para expresiones.
Validaciones explícitas	1	Regla semántica declarada en el DSL.
Restricciones de orden	1	Regla local must follow.

Tabla 6.2: Métricas de la definición textual .m2b de AppMaker

Métrica del caso AppMaker	Valor	Interpretación
Clases Ecore	40	Conceptos potencialmente convertibles en tipos de bloque o tipos abstractos de conexión.
Clases abstractas	3	Familias de bloques usadas para conexiones tipadas, no instanciables directamente.
Enumeraciones	11	Fuentes de campos desplegados.
Atributos	85	Campos visuales, valores por defecto y datos exportables.
Referencias totales	29	Relaciones estructurales y no estructurales del dominio.
Referencias de contención	21	Entradas de sentencias para bloques hijos.
Referencias no contenidas	8	Desplegados dinámicos hacia elementos existentes.
Anotaciones Ecore	221	Metadatos de sintaxis visual, interfaz, validación y código.
Reglas de validación generadas	70	Reglas publicadas en <code>validation_blocks.json</code> .
Objetos del modelo de ejemplo	58	Tamaño del sample cargado y exportado como JSON/XML.

Tabla 6.3: Métricas del metamodelo Ecore AppMaker y de la salida generada

6.3 AppMaker por las dos rutas de entrada

AppMaker representa una aplicación visual con páginas, componentes, acciones, navegación y fuentes de datos. El resultado generado es un editor de modelado: el usuario no recibe una aplicación final cerrada, sino un entorno visual para construir modelos de aplicaciones. El dominio incluye referencias cruzadas representativas: un `ListView` referencia una fuente de datos, y una acción `Navigate` referencia una página destino. También se usan plantillas de código para generar una representación textual de la aplicación.

La ruta Ecore parte de `model/app_maker.ecore`. La estructura del dominio se expresa como metamodelo EMF y la información específica de Blockly se añade mediante anotaciones. Las anotaciones con fuente `blockly` controlan aspectos de sintaxis visual, las anotaciones `ui` describen metadatos de interfaz, las anotaciones `code` definen plantillas de generación y las anotaciones `runtime` seleccionan opciones de ejecución.

La ruta textual parte de `examples/app_maker.m2b`. En este fichero se declaran categorías, clases, atributos, contenciones, referencias, entradas de valor, validaciones y opciones de presentación con una sintaxis más compacta que el XML/XMI de Ecore. El fichero `.m2b` no representa una instancia de una aplicación AppMaker, sino la definición del lenguaje de bloques AppMaker que se transforma a `EditorSpec`.

Para que el caso de estudio sea revisable, la memoria no se limita a describir AppMaker de forma narrativa. El caso se documenta con fragmentos de código de entrada, fragmentos de modelos generados y capturas de pantalla de los editores. Los Listados 5.2 y 5.5 muestran partes representativas del DSL textual `.m2b` y de la especificación Ecore evaluada. El Listado 6.1 muestra el modelo intermedio obtenido y el Listado 6.2 muestra un fragmento del generador JavaScript emitido para uno de los bloques del dominio. Las Figuras 5.2, 6.2, 5.1 y 6.3 complementan esos fragmentos con evidencia visual de las salidas generadas.

Las dos rutas producen una instancia del modelo intermedio `EditorSpec`. Esta instancia se serializa como XMI dentro de la carpeta `intermediate/` de la salida generada. A partir de ese modelo se generan los artefactos HTML/JavaScript, las validaciones, las páginas de edición, el informe de generación, el modelo de ejemplo y la vista Blockly de reglas de validación. Además, el editor generado permite exportar el modelo creado por el usuario como JSON y como XMI de instancia del dominio. Por lo tanto, en la evaluación se distinguen dos niveles: el XMI intermedio `EditorSpec`, usado por la cadena de generación, y el XMI de instancia, producido por el editor generado.

El Listado 6.1 muestra un fragmento del XMI generado por la ruta Ecore. Este artefacto permite inspeccionar el resultado de la transformación antes de generar JavaScript: las clases y relaciones del dominio ya aparecen como categorías, tipos de bloque, campos y entradas tipadas.

```
<blocklyspec:EditorSpec domainName="AppMaker"
  nsURI="http://www.example.org/appmaker"
  codeLanguage="javascript" codeFileExtension="js">
  <categories name="Pages" label="Pages" colour="260"/>
  <categories name="Components" label="Components" colour="160"/>
  <categories name="Data" label="Data" colour="210"/>
  <blockTypes typeName="App" label="App" colour="260"
    categoryName="Pages" connectionType="NONE">
    <fields name="name" defaultValue="Task Tracker" required="true"/>
    <statementInputs name="dataSources" checkType="DataSource"
      lowerBound="1" upperBound="20"/>
    <statementInputs name="pages" checkType="Page"
      lowerBound="1" upperBound="20"/>
  </blockTypes>
</blocklyspec:EditorSpec>
```

Listado 6.1: Fragmento del modelo intermedio XMI generado para AppMaker

El Listado 6.2 muestra un fragmento representativo del código JavaScript producido por el generador para AppMaker. Este código ilustra que la salida no se limita a pintar bloques: también se emiten funciones que recorren el workspace, leen campos y referencias, y serializan el modelo visual en una representación textual exportable.

```
javascript.javascriptGenerator.forBlock['ListView'] = function(block) {
  var emptyText = block.getFieldValue('emptyText');
  var source = block.getFieldValue('source');
  var itemTitle =
    javascript.javascriptGenerator.valueToCode(block, 'itemTitle', 0) || 'null';

  return '{' +
    '"_type": "ListView", ' +
    '"_blockId": ' + JSON.stringify(block.id) + ', ' +
    '"emptyText": ' + JSON.stringify(emptyText) + ', ' +
    '"source": ' + JSON.stringify(source) + ', ' +
    '"itemTitle": ' + itemTitle +
    ' },\n';
};
```

Listado 6.2: Fragmento legible del generador JavaScript emitido para ListView

La Figura 5.2 muestra el editor generado desde Ecore. Se ha cargado el modelo de ejem-

plo para comprobar visualmente que el editor incluye páginas, componentes, acciones, fuentes de datos, referencias y salida de modelo. La Figura 6.2 muestra la misma idea desde la ruta textual, usada para comprobar que el DSL `.m2b` puede describir el mismo lenguaje de bloques a nivel de autoría.

Además del editor final, la herramienta genera evidencias revisables. La Figura 5.1 muestra el informe producido para la ruta Ecore. El informe resume el mapeo desde la especificación de entrada hasta los bloques generados. La Figura 6.3 muestra el workspace visual de validaciones, donde las reglas generadas se representan también como bloques. Ese workspace permite descargar un `validation_blocks.edited.json`; el comando `Apply Validation Blocks to Source` aplica los cambios soportados de ese fichero a la fuente `.m2b` o `.ecore`.

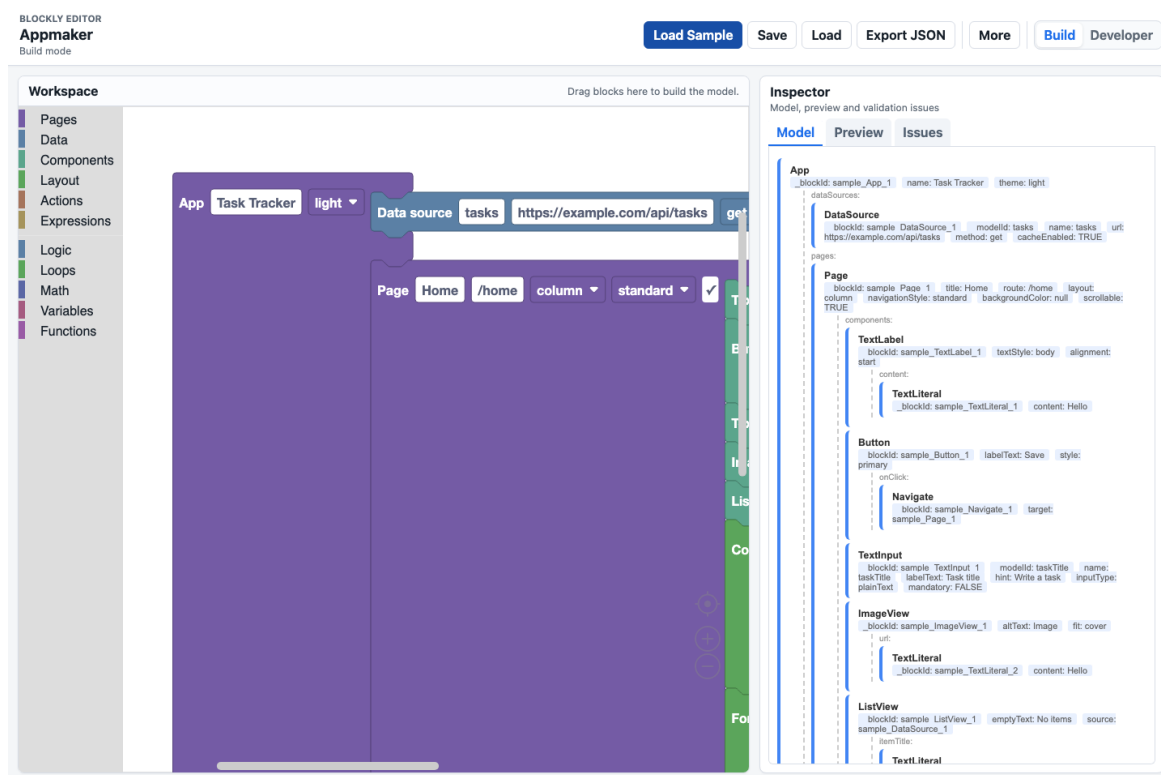


Figura 6.2: Editor Blockly generado a partir de app_maker .m2b

Appmaker Visual Validation Workspace

Copy JSON Import JSON **Download JSON** Download Workspace Download Runtime Download Source Snippets

Reset 70 rules

Rules
Logic
Accessors
Literals

```

rule Navigate_after_Alert
target Navigate type MUST_FOLLOW
message Navigate must follow Alert.
condition
    Navigate previous block is Alert

rule App_dataSources
target App type CARDINALITY
message App.dataSources has invalid cardinality.
condition
    App . dataSources children count
    min 1 max 20

rule App_pages
target App type CARDINALITY
message App.pages has invalid cardinality.
condition
    App . pages children count
    min 1 max 20

rule Page_components
target Page type CARDINALITY
message Page.components has invalid cardinality.
condition
    Page . components children count
    min 1 max 40

rule Page_onEnter
target Page type CARDINALITY
message Page.onEnter has invalid cardinality.
condition
    Page . onEnter children count
    min 0 max 10

rule Button_onClick
target Button type CARDINALITY
message Button.onClick has invalid cardinality.
condition
    Button . onClick children count
    min 0 max 10

rule Container_children

```

Rules

- Navigate_after_Alert
Navigate / MUST_FOLLOW
- App_dataSources
App / CARDINALITY
- App_pages
App / CARDINALITY
- Page_components
Page / CARDINALITY
- Page_onEnter
Page / CARDINALITY
- Button_onClick
Button / CARDINALITY
- Container_children
Container / CARDINALITY
- Card_children
Card / CARDINALITY
- Form_children
Form / CARDINALITY
- Form_onSubmit
Form / CARDINALITY
- TabView_panels
TabView / CARDINALITY

Figura 6.3: Workspace visual de reglas de validación generado para AppMaker

6.4 Cobertura funcional

La Tabla 6.4 resume qué capacidades se ejercitan en AppMaker. El objetivo no es afirmar que un único ejemplo cubra todos los dominios posibles, sino comprobar que el ejemplo conductor atraviesa las partes principales de la arquitectura.

La cobertura muestra que AppMaker no es solo una demostración visual. El ejemplo ejercita las dos rutas de entrada, el metamodelo intermedio, la generación HTML/JavaScript, las referencias, las validaciones, la exportación JSON/XMI, los informes y las plantillas de código. Esto apoya la idea de que Model2Blockly es una cadena generativa reutilizable, aunque una evaluación más amplia con otros metamodelos queda como trabajo futuro.

Capacidad	Evidencia en AppMaker
Ruta Ecore	model/app_maker.ecore con anotaciones.
Ruta textual .m2b	examples/app_maker.m2b con categorías, clases, referencias, contenciones, valores y validaciones.
Contención y cardinalidad	App contiene pantallas, Page contiene componentes y algunas relaciones tienen límites mínimos y máximos.
Herencia y clases abstractas	Component, Action y TextExpression son conceptos abstractos con subtipos concretos.
Referencias dinámicas	ListView.source, Navigate.target, SubmitForm.endpoint y SetInputValue.input se generan como despleguables dinámicos.
Entradas de valor y bloques de salida	Alert.message, ListView.itemTitle, ImageView.url y SetInputValue.newValue reciben TextExpression; TextLiteral, InputValue, DataField y JoinText se generan como bloques de salida.
Shadow blocks	Las entradas de valor usan TextLiteral como bloque sombra por defecto.
Validaciones de orden	La regla navigate_after_alert comprueba una restricción local de secuencia.
Campos avanzados	El dominio incluye enumeraciones, booleanos, valores por defecto y campos obligatorios.
Plantillas de código	Las clases principales incluyen plantillas para producir una representación JavaScript del modelo.
Exportación de instancia	El editor generado permite exportar el workspace como JSON y como XMI de instancia del dominio.
Trazabilidad de generación	La ruta Ecore produce XMI intermedio, generation_report.html, README.md, sample_model.json y ficheros de validación visual.
Validaciones visuales	Las reglas se publican como validation_blocks.json y pueden inspeccionarse en validation_workspace.html.
Metadatos de interfaz	Las anotaciones Ecore incluyen etiquetas, grupos, orden, widgets de campo y campos de referencia para personalizar la salida HTML.

Tabla 6.4: Cobertura funcional del ejemplo AppMaker

6.5 Verificaciones técnicas

Además del ejemplo conductor, se han realizado verificaciones técnicas sobre los artefactos generados y sobre el código del proyecto. La primera verificación consiste en comprobar que los ficheros JavaScript generados no contienen errores de sintaxis. Para ello se ha usado una comprobación sintáctica de Node.js sobre la salida generada. No se detectaron errores de sintaxis en los ficheros revisados, lo que aporta una comprobación reproducible sobre la corrección sintáctica de la salida JavaScript.

La segunda verificación es la existencia de pruebas JUnit por capa. El proyecto incluye 142 pruebas distribuidas en seis clases. La Tabla 6.5 resume su cobertura.

Capa probada	Número	Cobertura principal
Modelo intermedio	27	Tipos, valores por defecto, conexiones y estructura del EditorSpec.
Serialización y modelo generado	5	Carga del metamodelo BlocklyEditorSpec, XMI intermedio y modelo EMF generado.
Adaptador Ecore	39	Conversión desde Ecore, anotaciones, referencias, conexiones y validaciones.
Adaptador textual .m2b	6	Conversión desde la sintaxis textual, campos obligatorios, referencias y metadatos de código.
Generador HTML/JavaScript	60	Generación HTML/JavaScript, bloques, toolbox, validaciones, referencias y exportación.
Validación DSL	5	Diagnósticos del DSL antes de generar el editor.

Tabla 6.5: Pruebas automáticas incluidas en el proyecto

La organización de las pruebas sigue la misma división que la arquitectura. Si falla una prueba del adaptador Ecore, el problema está en la ruta de entrada. Si falla una prueba del generador HTML, el problema está en la salida. Si falla una prueba del modelo intermedio, puede haberse roto el contrato común entre capas. Esta trazabilidad ayuda a mantener el proyecto.

Además de las pruebas JUnit, el repositorio incluye verificaciones de entrega. El script `verify:plugin` comprueba metadatos de bundles, feature, update site, versión 1.0.9.qualifier, entorno JavaSE-21 y presencia de artefactos generados. Los scripts `verify:docs`, `verify:domain-xmi` y `verify:patch` cubren documentación, carga EMF del XMI de instancia exportado y el flujo de edición/aplicación de reglas de validación. También existe un *smoke test* de navegador para cargar los editores generados, tratado como verificación complementaria porque depende de disponer de Playwright en el entorno local.

6.6 Resultados frente a los objetivos

La Tabla 6.6 resume la relación entre los objetivos específicos y los resultados observados en el ejemplo conductor.

Objetivo	Evidencia aportada
OE1	AppMaker ejercita bloques, campos, categorías, conexiones, referencias, validaciones y exportación.
OE2	AppMaker se genera desde un metamodelo Ecore anotado.
OE3	AppMaker también se define mediante el DSL textual <code>.m2b</code> .
OE4	Las dos rutas se normalizan hacia el mismo contrato <code>EditorSpec</code> .
OE5	La cadena produce editores HTML/JavaScript con bloques, toolbox, validaciones, páginas de edición, exportación JSON/XMI e informes de generación.
OE6	El ejemplo incluye referencias dinámicas, validaciones, workspace visual de reglas, personalización de interfaz y plantillas de código.
OE7	La solución se contrasta con AppMaker por las dos rutas de entrada y con 142 pruebas automáticas organizadas por capa.

Tabla 6.6: Resultados de evaluación frente a los objetivos

La evaluación respalda la idea central del trabajo dentro del alcance definido: es viable generar editores Blockly desde descripciones de dominio de alto nivel y mantener una arquitectura común basada en un modelo intermedio. La evidencia principal no es una salida programada a mano, sino que AppMaker se transforma a `EditorSpec`, se serializa como XMI y desde ahí se genera el editor Blockly revisable.

6.7 Correspondencia entre metamodelo y editor Blockly

La evaluación no asume que un metamodelo Ecore y un editor Blockly tengan la misma expresividad. La relación entre ambos es una correspondencia práctica definida por el generador. Algunos elementos de Ecore se trasladan de forma directa: clases concretas a tipos de bloque, atributos a campos, contenciones a entradas de sentencias, referencias no contenidas a desplegables dinámicos y cardinalidades a reglas de validación. Otros elementos requieren decisiones de sintaxis concreta, como colores, etiquetas, categorías, widgets, disposición en línea o campos usados para mostrar referencias.

También existen elementos de Ecore que no tienen una representación Blockly equivalente completa en esta versión. Por ejemplo, las restricciones OCL arbitrarias, algunas relaciones bidireccionales complejas, la importación genérica de instancias XMI existentes, los tipos de datos personalizados con semántica rica o reglas globales sobre colecciones no se traducen de forma general. La herramienta cubre patrones frecuentes y genera advertencias, pero no convierte el editor Blockly en un editor EMF completo.

En sentido contrario, el editor Blockly contiene información que no pertenece al metamodelo abstracto: color, toolbox, tooltip, texto visible, orden de campos, shadow blocks, vista de validaciones, botones de exportación o preview HTML. Esa información describe una sintaxis concreta y una experiencia de edición, no la semántica estructural del dominio. Por ello se almacena como anotaciones o metadatos y no debe confundirse con el metamodelo de dominio.

La conclusión es que Model2Blockly busca una equivalencia útil para generación, no una equivalencia semántica total. El editor generado permite crear y exportar modelos que respetan la estructura principal del metamodelo dentro de las reglas soportadas, pero hay modelos Ecore válidos que pueden requerir extensiones del generador para editarse cómodamente en Blockly, y hay decisiones visuales de Blockly que no modifican la expresividad del metamodelo.

6.8 Limitaciones de la evaluación

La evaluación realizada tiene límites. En primer lugar, AppMaker ha sido diseñado por el autor del proyecto. Aunque cubre varias estructuras, no equivale a una evaluación con usuarios externos ni con modelos industriales de gran tamaño. En segundo lugar, las compro-

baciones generadas se implementan como advertencias y confirmaciones de exportación, no como un mecanismo formal de verificación semántica completa. En tercer lugar, la comprobación con `node --check` solo verifica sintaxis JavaScript, no interacción visual en todos los navegadores.

Además, la ruta Ecore se ha evaluado con un metamodelo representativo, pero no con un conjunto amplio de metamodelos heterogéneos. Funciones como la sincronización bidireccional completa en todos los runtimes, restricciones OCL arbitrarias, importación genérica de XMI o generación de código con análisis semántico avanzado quedan fuera del alcance actual. La edición visual de validaciones también debe interpretarse de forma acotada: solo se aplican de vuelta a la fuente las reglas que tienen una traducción directa.

Estas limitaciones no invalidan la evidencia técnica, pero sí acotan su interpretación. Model2Blockly aporta evidencia de una cadena generativa funcional y reutilizable en el caso estudiado. Una evaluación futura podría incluir más metamodelos, medir el esfuerzo frente a una implementación manual en Blockly y realizar estudios con usuarios finales.

6.9 Conclusión de la evaluación

El ejemplo conductor y las verificaciones técnicas apoyan la idea central del TFM. Model2Blockly no genera un editor escrito manualmente, sino un editor derivado de descripciones de dominio. La ruta Ecore permite generar desde metamodelos. La ruta textual `.m2b` permite definir lenguajes de bloques de forma compacta. El modelo intermedio permite mantener el contrato entre transformación y generación. La disponibilidad del update site p2, las demos desplegadas y los informes de generación complementan esa evidencia al mostrar que la propuesta puede instalarse, ejecutarse y revisarse fuera del entorno de desarrollo del autor.

Capítulo 7

Conclusiones y trabajo futuro

Este Trabajo Fin de Máster ha tratado la creación de lenguajes de dominio específico con sintaxis basada en bloques. La motivación parte de un problema práctico: Blockly facilita la creación de editores visuales, pero cada dominio todavía requiere definir a mano bloques, campos, toolbox, conexiones, validaciones y generadores. El trabajo propone una alternativa basada en ingeniería de lenguajes y modelos: describir el dominio a alto nivel y generar automáticamente el editor Blockly.

El capítulo se organiza en torno a los resultados y las perspectivas de la propuesta. En primer lugar, se analiza la viabilidad del enfoque y se resumen sus principales aportaciones. A continuación, se revisan las conclusiones obtenidas para cada objetivo y se presentan las limitaciones identificadas. Después se proponen posibles líneas de trabajo futuro y, finalmente, se ofrece un cierre global del trabajo realizado.

7.1 Viabilidad del enfoque

El trabajo desarrollado aporta evidencia de que es viable diseñar e implementar un entorno para la creación de lenguajes de dominio específico con sintaxis basada en bloques. Model2Blockly acepta metamodelos Ecore enriquecidos con anotaciones y modelos textuales .m2b definidos con Xtext. Ambas rutas se transforman en un modelo intermedio común, y desde él se generan editores Blockly ejecutables. La herramienta se materializa además como plugin Eclipse instalable mediante un update site p2, por lo que el enfoque no queda limitado a un prototipo ejecutado manualmente desde el código fuente.

La solución aprovecha información que ya existe en una descripción de dominio. Las clases se convierten en tipos de bloque, los atributos en campos, las contenciones en entradas estructurales, las referencias en despleables dinámicos y las cardinalidades en comprobaciones. Las anotaciones añaden metadatos de presentación. Este proceso reduce la programación manual en JavaScript y sigue la idea central de la ingeniería dirigida por modelos: utilizar modelos para especificar y generar software [5, 6].

El trabajo también refuerza la utilidad de separar lo que el dominio significa de la forma en que se muestra al usuario. El dominio se describe mediante Ecore o mediante el DSL textual .m2b; Blockly actúa como la representación visual generada para manipular ese dominio. Esta separación es coherente con la forma en que los lenguajes de dominio específico organizan sus conceptos y sus notaciones [16].

7.2 Principales aportaciones

La primera aportación es una arquitectura generativa que conecta Ecore, .m2b, EditorSpec y Blockly. La arquitectura introduce un modelo intermedio, en lugar de acoplar directamente cada entrada a una salida concreta. Las dos rutas se normalizan hacia el mismo contrato y no requieren generadores separados.

La segunda aportación es el modelo intermedio común. Este modelo recoge los elementos necesarios para construir un editor visual: bloques, campos, categorías, entradas de valor, entradas de sentencias, referencias, validaciones, opciones de workspace y metadatos de código. También define el contrato entre adaptadores y generadores, se serializa como XMI y se recarga antes de generar la salida final.

La tercera aportación es la ruta Ecore. El adaptador desarrollado permite partir de un metamodelo y derivar una especificación de editor. Esta ruta cubre la parte metamodelo-céntrica del sistema: usar un metamodelo, enriquecido cuando sea necesario con anotaciones, para generar código basado en Blockly.

La cuarta aportación es la ruta textual .m2b. Esta ruta facilita definir lenguajes de bloques de forma compacta, con una sintaxis cercana a los conceptos que después se convertirán en bloques. También incorpora validación estática mediante Xtext, lo que permite detectar errores antes de generar el editor cuando se usa esta forma de autoría.

La quinta aportación es la generación de editores ejecutables. La salida HTML/JavaScript produce definiciones de bloques, toolbox, generadores, validaciones, páginas HTML auto-

contenidas, exportación JSON/XMI, informes de generación, modelos de ejemplo y una vista Blockly de las reglas de validación.

La sexta aportación es la integración práctica de la herramienta. El plugin Eclipse ofrece comandos para generar desde ficheros `.m2b` y `.ecore`, y para aplicar de vuelta a la fuente cambios soportados en las reglas visuales de validación. El update site y las demos publicadas facilitan la revisión externa del resultado.

Finalmente, el trabajo aporta una evaluación basada en un ejemplo conductor y pruebas automáticas. AppMaker se describe como metamodelo Ecore anotado y como fichero textual `.m2b`, lo que permite comparar las dos formas de autoría. Las 142 pruebas JUnit organizadas por capa aportan evidencia adicional sobre el comportamiento del modelo intermedio, los adaptadores y los generadores.

7.3 Conclusiones por objetivos

Los objetivos específicos definidos al inicio del trabajo se han abordado de la siguiente forma:

- **OE1.** Se analizaron los elementos necesarios para un editor de bloques y se representaron explícitamente en el modelo intermedio: tipos de bloque, campos, conexiones, categorías, referencias, validaciones y generación.
- **OE2.** Se implementó una ruta Ecore que transforma clases, atributos, referencias, herencia, cardinalidades y anotaciones en una especificación Blockly.
- **OE3.** Se definió una ruta textual `.m2b` con Xtext para describir lenguajes de bloques sin editar directamente ficheros Ecore en ejemplos o configuraciones compactas.
- **OE4.** Se diseñó e implementó un modelo intermedio común como representación común independiente de la fuente de entrada.
- **OE5.** Se generaron editores HTML/JavaScript ejecutables con bloques, toolbox, validaciones, guardado, carga, exportación JSON/XMI e informes de generación.
- **OE6.** Se incorporaron comprobaciones, referencias dinámicas, personalización visual, metadatos de interfaz, plantillas de código y una vista Blockly editable para reglas de validación.

- **OE7.** Se evaluó la solución con AppMaker por la ruta Ecore y por la ruta textual .m2b, junto con pruebas automáticas sobre las capas principales del sistema.

Por lo tanto, el enfoque es técnicamente viable dentro del alcance definido. Un dominio descrito con Ecore anotado o con .m2b puede transformarse en un editor de bloques mediante una cadena generativa reproducible. La ruta textual muestra además que esa información puede escribirse de forma más compacta y normalizarse hacia el mismo modelo intermedio.

7.4 Limitaciones

Aunque los resultados respaldan el enfoque principal, el trabajo tiene limitaciones. La primera es que la evaluación usa un ejemplo diseñado dentro del proyecto. AppMaker cubre estructuras variadas, pero no equivale a una validación industrial con muchos metamodelos externos. Por ello, el caso estudiado apoya la viabilidad de la herramienta, pero no prueba su generalidad de forma exhaustiva.

La segunda limitación afecta a las validaciones. Model2Blockly genera advertencias para mínimos y máximos de bloques, campos obligatorios, referencias obligatorias, unicidad, reglas simples de orden, expresiones declaradas en anotaciones y un subconjunto sencillo de invariantes OCL. Este soporte cubre restricciones frecuentes, pero no equivale a un lenguaje completo de restricciones como OCL [19]. No se soportan invariantes arbitrarios, cuantificadores, navegación compleja ni análisis semántico profundo. Además, las comprobaciones generadas producen advertencias, no una validación bloqueante. La edición visual de reglas tampoco es una transformación bidireccional general: solo se aplican de vuelta a la fuente los cambios que tienen una correspondencia clara.

La tercera limitación se refiere a las referencias. Las referencias no contenidas se representan mediante despleables dinámicos, lo que permite seleccionar elementos existentes del workspace. Además, el runtime HTML sincroniza referencias opuestas no contenidas cuando ambos extremos son editables. La limitación actual está en los casos más complejos: referencias opuestas con semántica de contenedor y visualización avanzada de relaciones entre elementos.

La cuarta limitación está relacionada con la generación de código. La solución incluye plantillas de código y una representación textual por defecto, pero no incorpora todavía

formateadores específicos, análisis de tipos avanzado, gestión de imports, generación multiarchivo ni integración directa con herramientas externas de construcción o ejecución.

La quinta limitación es de evaluación de usuario. Este TFM no mide si los editores generados reducen efectivamente el esfuerzo de aprendizaje o mejoran la experiencia de usuarios no técnicos. La literatura sobre programación por bloques muestra que la modalidad de interacción influye en las prácticas de los usuarios novatos [2], pero validar empíricamente los editores generados requeriría un estudio específico.

7.5 Trabajo futuro

A partir de estas limitaciones se identifican varias líneas de trabajo futuro.

En primer lugar, sería útil ampliar el soporte de validación. Una opción sería integrar un lenguaje de restricciones más expresivo o un mecanismo de reglas declarativas. Así se podrían describir condiciones más complejas que los mínimos, máximos y reglas de orden inmediato. Esta línea también permitiría ampliar la traducción entre reglas visuales editadas y su representación en .m2b o en anotaciones Ecore.

En segundo lugar, la gestión de referencias podría evolucionar hacia un soporte más completo de referencias bidireccionales. Esto implicaría extender la sincronización de referencias opuestas en el runtime HTML, cubrir casos de contenedor, detectar referencias que apuntan a elementos ya eliminados y ofrecer visualizaciones más claras de las relaciones entre elementos del workspace.

En tercer lugar, se podría profundizar en la importación y exportación de modelos. Actualmente la herramienta se centra en generar editores y exportar el contenido del workspace. Una línea futura consistiría en importar modelos XMI existentes y reconstruir automáticamente el workspace Blockly correspondiente. Esto acercaría más la herramienta a flujos completos de ingeniería de modelos.

En cuarto lugar, la generación de código podría enriquecerse. Las plantillas actuales son suficientes para demostrar una ruta genérica de exportación, pero podrían ampliarse con validación de plantillas, tipado, formateadores, generación multiarchivo y objetivos específicos para lenguajes o plataformas concretas.

En quinto lugar, podrían explorarse interfaces web más avanzadas sobre el mismo modelo intermedio. Un futuro objetivo de generación podría incorporar un inspector estructurado, temas visuales por dominio, componentes personalizados y modos de edición más adaptados

a usuarios finales.

En sexto lugar, sería conveniente realizar una evaluación empírica con usuarios. Un estudio futuro podría comparar el esfuerzo necesario para construir un editor Blockly manualmente frente al uso de Model2Blockly, o analizar cómo interactúan usuarios no técnicos con editores generados para distintos dominios. Esta evaluación permitiría complementar la evidencia técnica de este TFM con datos de uso.

Por último, convendría robustecer el ciclo de publicación del plugin. La versión actual ya dispone de update site p2 y verificaciones de empaquetado, pero una línea futura sería automatizar más la publicación, el versionado y las pruebas de instalación sobre instalaciones limpias de Eclipse.

7.6 Cierre

Model2Blockly aporta evidencia de que es posible conectar técnicas de ingeniería de lenguajes con entornos de programación basada en bloques. El resultado no es un único editor visual, sino una infraestructura que transforma descripciones de dominio en editores Blockly ejecutables. El conocimiento del dominio se mantiene en modelos o DSLs de alto nivel, y la sintaxis visual se genera a partir de ellos.

El trabajo muestra que la combinación de Ecore, Xtext, un modelo intermedio EMF, generación HTML/JavaScript y Blockly es viable para construir editores de lenguajes de dominio específico con sintaxis basada en bloques. Dentro del alcance definido, esta combinación es una alternativa razonable a la programación manual de editores Blockly y deja una base extensible para trabajos futuros.

Bibliografía

- [1] Y. Lin y D. Weintrop, «The Landscape of Block-Based Programming: Characteristics of Block-Based Environments and How They Support the Transition to Text-Based Programming», *Journal of Computer Languages*, vol. 67, pág. 101 075, 2021. DOI: 10.1016/j.col.2021.101075
- [2] D. Weintrop y U. Wilensky, «How Block-Based, Text-Based, and Hybrid Block/Text Modalities Shape Novice Programming Practices», *International Journal of Child-Computer Interaction*, vol. 17, págs. 83-92, 2018. DOI: 10.1016/j.ijcci.2018.04.005
- [3] «What is Blockly?», Blockly, visitado 3 de jun. de 2026. dirección: <https://docs.blockly.com/guides/get-started/what-is-blockly/>
- [4] «Blockly Documentation», Blockly, visitado 3 de jun. de 2026. dirección: <https://docs.blockly.com/>
- [5] M. Brambilla, J. Cabot y M. Wimmer, *Model-Driven Software Engineering in Practice*, 2ª ed. Morgan & Claypool, 2017. DOI: 10.2200/S00751ED2V01Y201701SWE004
- [6] J. Hutchinson, J. Whittle y M. Rouncefield, «Model-driven engineering practices in industry: Social, organizational and managerial factors that lead to success or failure», *Science of Computer Programming*, vol. 89, págs. 144-161, 2014. DOI: 10.1016/j.scico.2013.03.017
- [7] D. Akdur, V. Garousi y O. Demirörs, «A Survey on Modeling and Model-Driven Engineering Practices in the Embedded Software Industry», *Journal of Systems Architecture*, vol. 91, págs. 62-82, 2018. DOI: 10.1016/j.sysarc.2018.09.007

- [8] «The Eclipse Modeling Framework Overview», Eclipse Foundation, visitado 3 de jun. de 2026. dirección: <https://help.eclipse.org/latest/topic/org.eclipse.emf.doc/references/overview/EMF.html>
- [9] D. Steinberg, F. Budinsky, M. Paternostro y E. Merks, *EMF: Eclipse Modeling Framework*, 2ª ed. Addison-Wesley Professional, 2008.
- [10] «Eclipse Xtext», Eclipse Foundation, visitado 3 de jun. de 2026. dirección: <https://projects.eclipse.org/projects/modeling.xtext>
- [11] L. Bettini, *Implementing Domain Specific Languages with Xtext and Xtend*, 2ª ed. Packt Publishing, 2016.
- [12] M. Resnick et al., «Scratch: Programming for All», *Communications of the ACM*, vol. 52, nº 11, págs. 60-67, 2009. DOI: 10.1145/1592761.1592779
- [13] J. Moreno-León y G. Robles, «Dr. Scratch: A Web Tool to Automatically Evaluate Scratch Projects», en *Proceedings of the Workshop in Primary and Secondary Computing Education*, ép. WiPSCE '15, ACM, 2015, págs. 132-133. DOI: 10.1145/2818314.2818338
- [14] «Getting Started with Blockly», Blockly, visitado 12 de jul. de 2026. dirección: <https://blocklycodelabs.dev/codelabs/getting-started/index.html>
- [15] «Blockly Documentation», Google Developers, visitado 4 de jun. de 2026. dirección: <https://developers.google.com/blockly>
- [16] B. Sal, D. García-Saiz, A. de la Vega y P. Sánchez, «Domain-Specific Languages for the Automated Generation of Datasets for Industry 4.0 Applications», *Journal of Industrial Information Integration*, vol. 41, pág. 100657, 2024. DOI: 10.1016/j.jii.2024.100657
- [17] A. Wąsowski y T. Berger, *Domain-Specific Languages: Effective Modeling, Automation, and Reuse*. Springer Cham, 2023. DOI: 10.1007/978-3-031-23669-3
- [18] «Unified Modeling Language Specification», Object Management Group, visitado 4 de jun. de 2026. dirección: <https://www.omg.org/spec/UML/>
- [19] «Object Constraint Language Specification», Object Management Group, visitado 4 de jun. de 2026. dirección: <https://www.omg.org/spec/OCL/>

- [20] J. Luoma, S. Kelly y J.-P. Tolvanen, «Defining Domain-Specific Modeling Languages: Collected Experiences», en *Proceedings of the 4th OOPSLA Workshop on Domain-Specific Modeling*, ép. Computer Science and Information Systems Reports, University of Jyväskylä, 2004. dirección: <https://www.dsmforum.org/events/dsm04/luoma.pdf>
- [21] L. Bettini, «Implementing Type Systems for the IDE with Xsemantics», *Journal of Logical and Algebraic Methods in Programming*, vol. 85, nº 5, págs. 655-680, 2016. DOI: 10.1016/j.jlamp.2015.11.005
- [22] «Xtend Documentation», Eclipse Foundation, visitado 12 de jul. de 2026. dirección: <https://eclipse.dev/Xtext/xtend/documentation/>
- [23] E. Pasternak, R. Fenichel y A. N. Marshall, «Tips for Creating a Block Language with Blockly», en *2017 IEEE Blocks and Beyond Workshop*, ép. B&B, IEEE, 2017, págs. 21-24. DOI: 10.1109/BLOCKS.2017.8120404
- [24] M. V. Merino y T. van der Storm, «Language Workbench Support for Block-Based DSLs», en *BLOCKS+ Proceedings*, 2018. dirección: <https://homepages.cwi.nl/~storm/publications/blocks18.pdf>
- [25] M. V. Merino y T. van der Storm, «Block-Based Syntax from Context-Free Grammars», en *Proceedings of the 13th ACM SIGPLAN International Conference on Software Language Engineering*, ép. SLE 2020, ACM, 2020, págs. 283-295. DOI: 10.1145/3426425.3426948
- [26] M. V. Merino y K. van Wijk, «Workbench for Creating Block-Based Environments», en *Proceedings of the 15th ACM SIGPLAN International Conference on Software Language Engineering*, ép. SLE 2022, ACM, 2022, págs. 61-73. DOI: 10.1145/3567512.3567518